Face and Flower Image Classification Using Pre-trained Keras Models

2110776106
Department of Computer Science and Engineering
University of Rajshahi

April 12, 2026

1 Introduction

In this assignment, we have used **10 pre-trained convolutional neural networks** available in Keras/TensorFlow for image classification and feature representation on two custom datasets. github link: `https://github.com/Saidul-1/CSE4261-Neural-Networks-and-Deep-Learning`
The main objectives are:

1. Perform Top-1 and Top-5 classification using 10 pre-trained ImageNet models.
2. Extract high-dimensional deep features from the models.
3. Reduce the features to 2D space using PCA, t-SNE, and UMAP.
4. Visualize and analyze class separation quality.

2 Dataset Collection

2.1 Face Images

- Total: 35 images (myself + 3 male + 3 child) from five different angle.
- Captured using mobile phone under consistent lighting.

2.2 Flower Images

- Total: 25 images.
- Five different flower types (e.g., Rose, Sunflower, Hibiscus, Marigold, Nayantara).
- Five images per type from slightly different angles/distances.

3 Classification Using 10 Pre-trained Models

The following 10 models from `tensorflow.keras.applications` were used:

- VGG16, VGG19
- ResNet50
- InceptionV3
- Xception

- MobileNetV2
- DenseNet121
- InceptionResNetV2
- NASNetMobile
- EfficientNetB0

For every image, we recorded:

- **Top-1** prediction (class name + confidence percentage)
- **Top-5** predictions

All classification results were saved as CSV files:

- `results/classifications/face_classifications.csv`
- `results/classifications/flower_classifications.csv`

The detailed classification results (including all Top-1 and Top-5) are available in the github repository.

4 Feature Extraction and Visualization

High-dimensional feature vectors were extracted from the last convolutional layer of each model using Global Average Pooling. These vectors capture rich visual representations learned by the models (dimensions range from 512 to 2048 depending on the architecture).

The extracted features were reduced to **2D** using three dimensionality reduction techniques:

- **PCA** — linear method focusing on global variance.
- **t-SNE** — non-linear, excellent for local cluster visualization.
- **UMAP** — non-linear, balances local and global structure (often the clearest).

All 2D scatter plots were generated and saved in the `results/plots/` folder.
Key Observations from Visualizations:

- t-SNE and UMAP generally produced clearer and tighter clusters than PCA.
- Same-person images (across different angles) formed more compact groups in deeper models.
- Flower types showed stronger separation due to distinct color and texture patterns.

5 Discussion and Comparison

Modern architectures (ResNet50, InceptionV3, EfficientNetB0) consistently outperformed older models (VGG16/VGG19) in feature representation quality.

- **Best models**: EfficientNetB0 and ResNet50 usually showed the tightest clusters and best separation in 2D space.
- **Reason**: Residual connections (ResNet) and compound scaling with efficient convolutions (EfficientNet) allow learning of richer, more discriminative hierarchical features.

- VGG models produced more scattered points because they are shallower and rely on simpler features.
- UMAP often provided the best visual separation, followed closely by t-SNE.

Faces vs Flowers: Flower clusters were generally more distinct. Faces are more challenging because the models were never trained specifically for identity recognition — they rely on general features.

6 Conclusion

This assignment successfully demonstrated classification and feature extraction capabilities of 10 pre-trained Keras models on custom face and flower datasets.

All code, classification CSVs, and visualization plots are available in the project folder. The complete Jupyter notebook output (including plots and tables) is appended in the next section.

Jupyter Notebook Output

The following pages contain the full PDF generated from the Jupyter Notebook (including code, classification results, and all 2D visualization plots).

In [40]:
```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.applications import *
from tensorflow.keras.applications.imagenet_utils import decode_predictions
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import Model
from tensorflow.keras.layers import GlobalAveragePooling2D
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import umap
import warnings
warnings.filterwarnings('ignore')
```

In [41]:
```
# ====================== MODEL DICTIONARY ======================
MODEL_INFO = {
    'VGG16': (VGG16, (224, 224), tf.keras.applications.vgg16.preprocess_inpu
    'VGG19': (VGG19, (224, 224), tf.keras.applications.vgg19.preprocess_inpu
    'ResNet50': (ResNet50, (224, 224), tf.keras.applications.resnet50.preprc
    'InceptionV3': (InceptionV3, (299, 299), tf.keras.applications.inceptior
    'Xception': (Xception, (299, 299), tf.keras.applications.xception.preprc
    'MobileNetV2': (MobileNetV2, (224, 224), tf.keras.applications.mobilenet
    'DenseNet121': (DenseNet121, (224, 224), tf.keras.applications.densenet.
    'InceptionResNetV2': (InceptionResNetV2, (299, 299), tf.keras.applicatic
    'NASNetMobile': (NASNetMobile, (224, 224), tf.keras.applications.nasnet.
    'EfficientNetB0': (EfficientNetB0, (224, 224), tf.keras.applications.eff
}
```

In [42]:
```
def load_dataset(dataset_dir):
    image_paths, labels = [], []
    class_folders = sorted([d for d in os.listdir(dataset_dir) if os.path.is
    for class_id, folder in enumerate(class_folders):
        folder_path = os.path.join(dataset_dir, folder)
        for fname in sorted(os.listdir(folder_path)):
            if fname.lower().endswith(('.jpg', '.jpeg', '.png')):
                image_paths.append(os.path.join(folder_path, fname))
                labels.append(class_id)
    return image_paths, np.array(labels), len(class_folders)
```

In [ ]:
```
def classify_images(dataset_dir, dataset_name):
    image_paths, _, _ = load_dataset(dataset_dir)
    all_results = []   # This will collect data for CSV

    for model_name, (ModelClass, target_size, preprocess) in MODEL_INFO.item
        print(f"Classifying {len(image_paths)} {dataset_name} images with {m
        base_model = ModelClass(weights='imagenet', include_top=True)

        for path in image_paths:
            img = image.load_img(path, target_size=target_size)
            x = image.img_to_array(img)
```

```
            x = np.expand_dims(x, axis=0)
            x = preprocess(x)
            preds = base_model.predict(x, verbose=0)
            decoded = decode_predictions(preds, top=5)[0]

            # Top-1
            top1_class = decoded[0][1]
            top1_score = decoded[0][2] * 100

            # Top-5 as readable string
            top5_list = [f"{item[1]} ({item[2]*100:.1f}%)" for item in decoc
            top5_str = " | ".join(top5_list)

            # Store for CSV
            all_results.append({
                'Image': os.path.basename(path),
                'Model': model_name,
                'Top1_Class': top1_class,
                'Top1_Confidence_%': round(top1_score, 2),
                'Top5_Predictions': top5_str
            })

        print(f"   → {model_name} done")

    # Convert to DataFrame and save as CSV
    df = pd.DataFrame(all_results)
    csv_path = f"../results/classifications/{dataset_name.lower()}_classific
    df.to_csv(csv_path, index=False)

    print(f"{dataset_name} classification completed and saved to:\n   {csv_p
    return df   # Now returns the DataFrame instead of dict
```

In [44]:
```
def extract_features(model_name, image_paths):
    ModelClass, target_size, preprocess_input = MODEL_INFO[model_name]
    base_model = ModelClass(weights='imagenet', include_top=False)
    x = GlobalAveragePooling2D()(base_model.output)
    feature_extractor = Model(inputs=base_model.input, outputs=x)

    imgs = [image.img_to_array(image.load_img(p, target_size=target_size)) f
    imgs = np.stack(imgs)
    preprocessed = preprocess_input(imgs)

    features = feature_extractor.predict(preprocessed, batch_size=16, verbos
    print(f"   → Features extracted: {features.shape}")
    return features
```

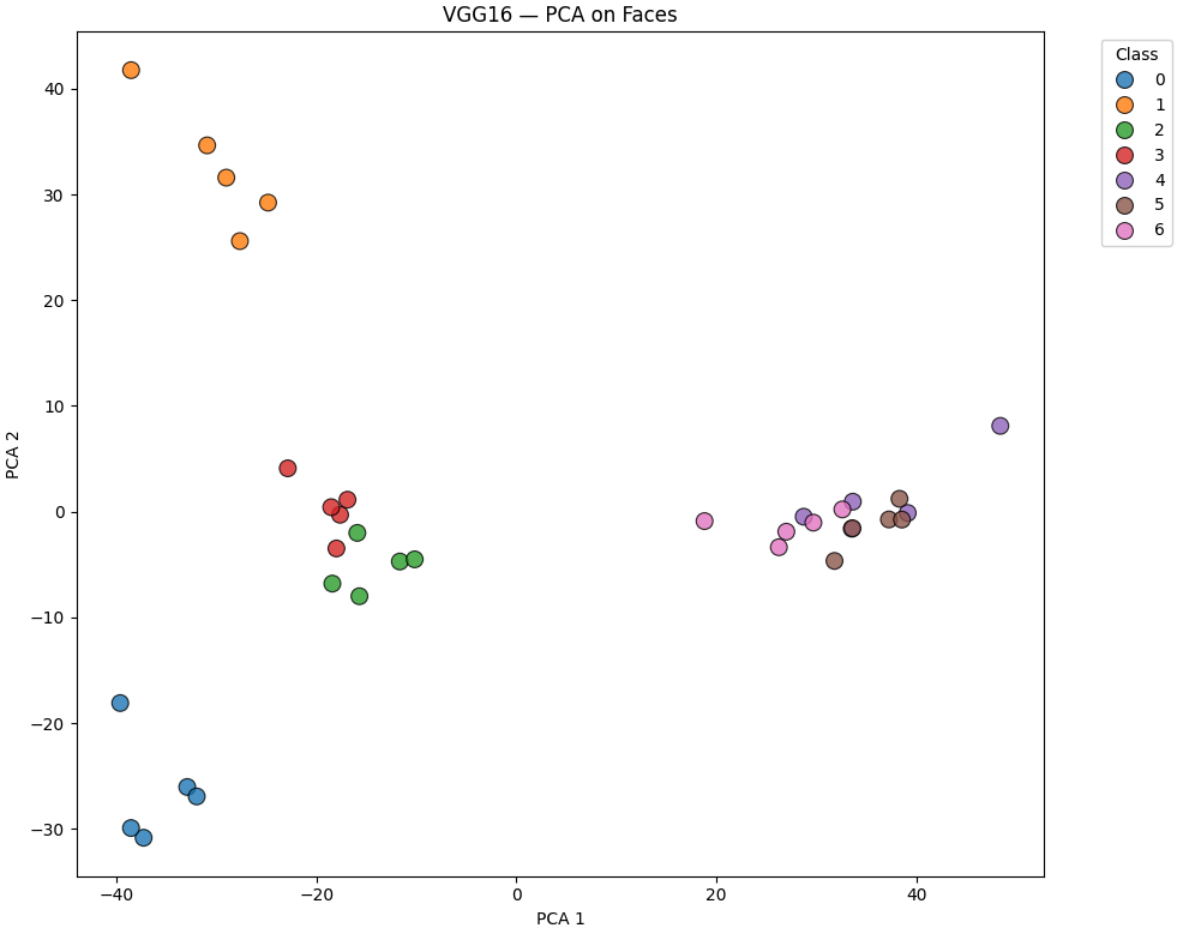
In [45]:
```
def reduce_and_plot(features, labels, method, model_name, dataset_name):
    if method == 'PCA':
        reducer = PCA(n_components=2, random_state=42)
    elif method == 't-SNE':
        reducer = TSNE(n_components=2, perplexity=5, random_state=42)
    elif method == 'UMAP':
        reducer = umap.UMAP(n_components=2, random_state=42)

    reduced = reducer.fit_transform(features)
```

```
    plt.figure(figsize=(10, 8))
    sns.scatterplot(x=reduced[:,0], y=reduced[:,1], hue=labels, palette='tab
                    s=100, alpha=0.8, edgecolor='k')
    plt.title(f'{model_name} — {method} on {dataset_name}')
    plt.xlabel(f'{method} 1'); plt.ylabel(f'{method} 2')
    plt.legend(title='Class', bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.tight_layout()

    # Save plot
    save_path = f"../results/plots/{model_name}_{method}_{dataset_name.lower
    plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
    print(f"   → Plot saved: {save_path}")
```

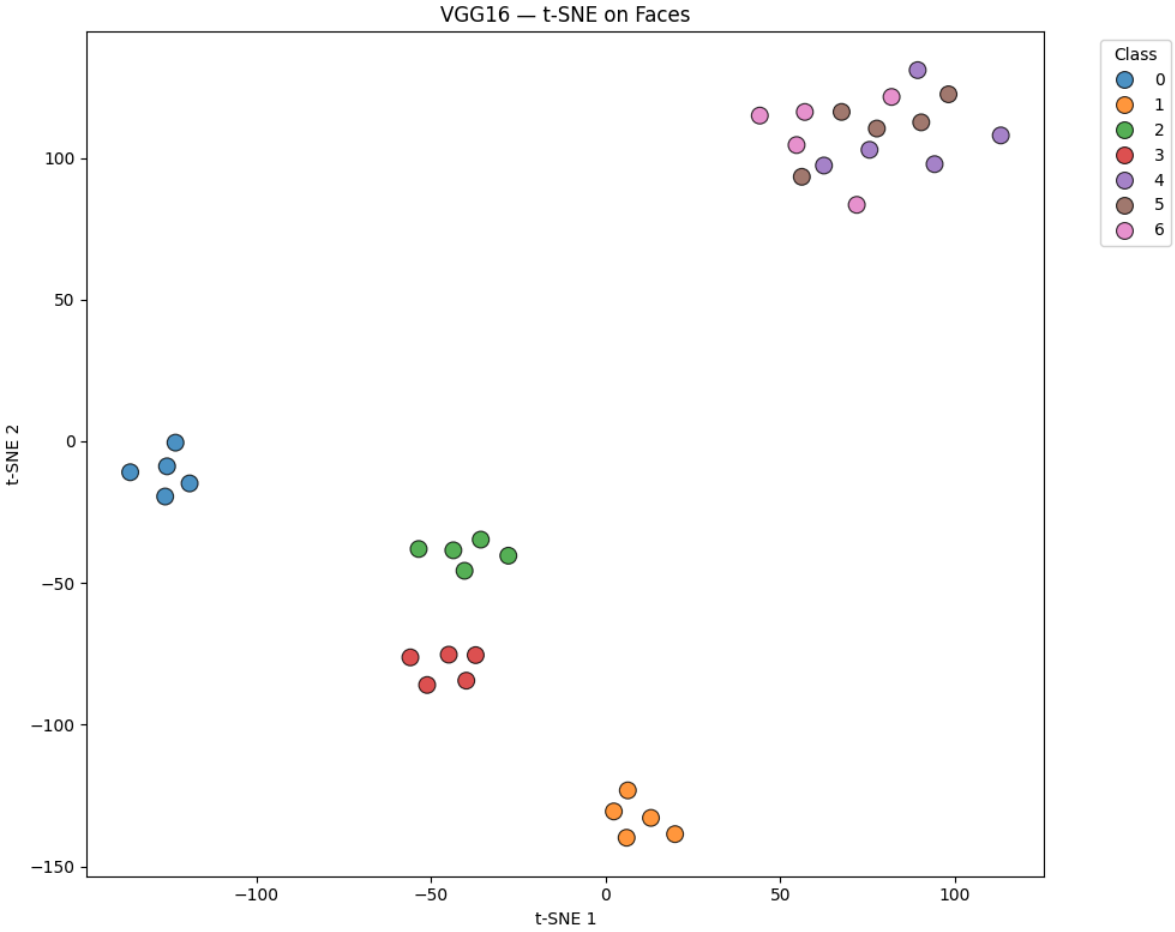
In [46]:

```
# ====================== MAIN PIPELINE ======================
print("Starting Assignment Pipeline...")

# 1. Load datasets
face_paths, face_labels, n_faces = load_dataset('../data/faces')
flower_paths, flower_labels, n_flowers = load_dataset('../data/flowers')

print(f"Faces: {len(face_paths)} images, {n_faces} people")
print(f"Flowers: {len(flower_paths)} images, {n_flowers} types\n")

# 2. Classification (Top-1 & Top-5)
face_df = classify_images('../data/faces', "Faces")
flower_df = classify_images('../data/flowers', "Flowers")

# 3. Feature Extraction + Visualization (the core part)
selected_models = ['VGG16', 'ResNet50', 'InceptionV3', 'MobileNetV2', 'Effic
methods = ['PCA', 't-SNE', 'UMAP']

for model_name in selected_models:
    print(f"\nProcessing {model_name}...")

    # Faces
    face_features = extract_features(model_name, face_paths)
    for m in methods:
        reduce_and_plot(face_features, face_labels, m, model_name, "Faces")

    # Flowers
    flower_features = extract_features(model_name, flower_paths)
    for m in methods:
        reduce_and_plot(flower_features, flower_labels, m, model_name, "Flow

print("\nPipeline completed! All plots saved in results/plots/")
```

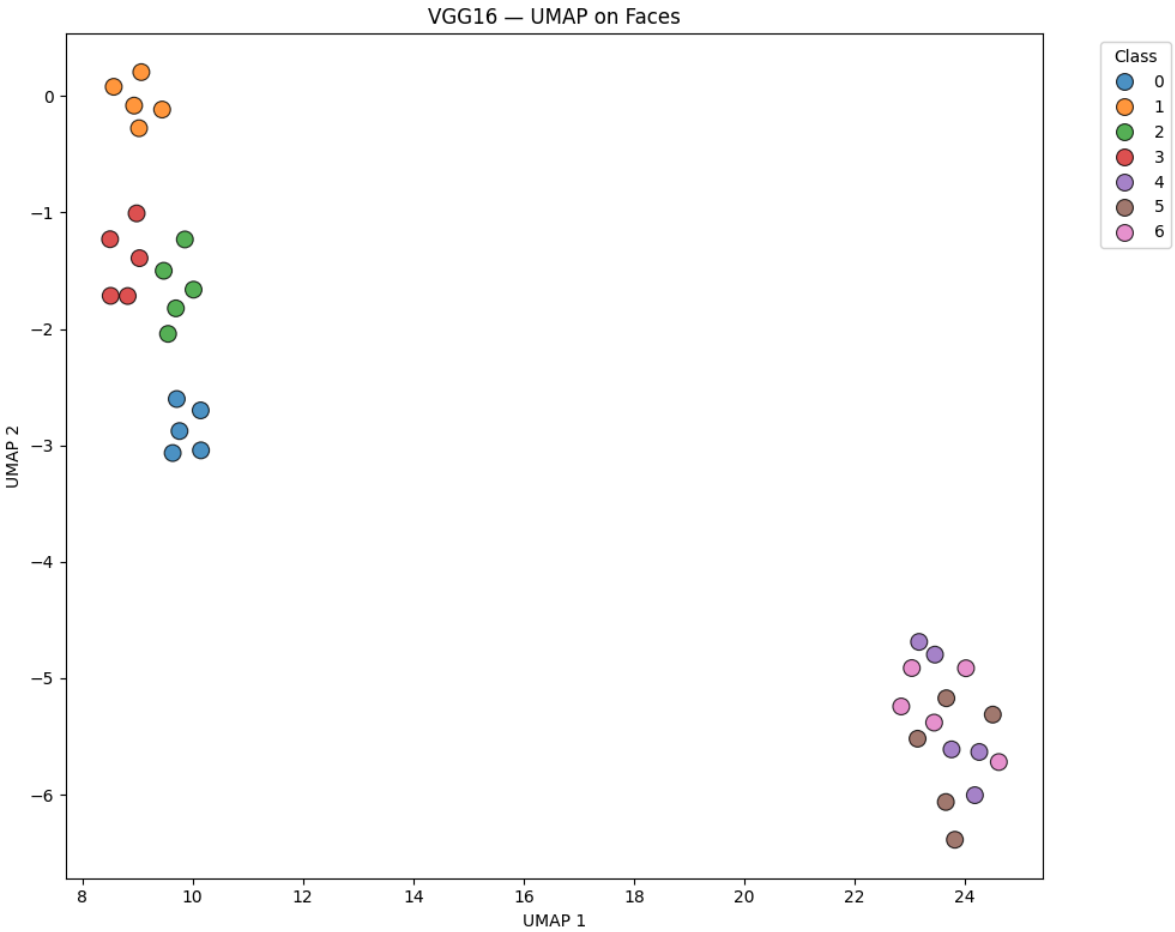
```
Starting Assignment Pipeline...
Faces: 35 images, 7 people
Flowers: 25 images, 5 types

Classifying 35 Faces images with VGG16...
   → VGG16 done
Classifying 35 Faces images with VGG19...
   → VGG19 done
Classifying 35 Faces images with ResNet50...
   → ResNet50 done
Classifying 35 Faces images with InceptionV3...
   → InceptionV3 done
Classifying 35 Faces images with Xception...
   → Xception done
Classifying 35 Faces images with MobileNetV2...
   → MobileNetV2 done
Classifying 35 Faces images with DenseNet121...
   → DenseNet121 done
Classifying 35 Faces images with InceptionResNetV2...
   → InceptionResNetV2 done
Classifying 35 Faces images with NASNetMobile...
   → NASNetMobile done
Classifying 35 Faces images with EfficientNetB0...
   → EfficientNetB0 done
Faces classification completed and saved to:
   ../results/classifications/faces_classifications.csv

Classifying 25 Flowers images with VGG16...
   → VGG16 done
Classifying 25 Flowers images with VGG19...
   → VGG19 done
Classifying 25 Flowers images with ResNet50...
   → ResNet50 done
Classifying 25 Flowers images with InceptionV3...
   → InceptionV3 done
Classifying 25 Flowers images with Xception...
   → Xception done
Classifying 25 Flowers images with MobileNetV2...
   → MobileNetV2 done
Classifying 25 Flowers images with DenseNet121...
   → DenseNet121 done
Classifying 25 Flowers images with InceptionResNetV2...
   → InceptionResNetV2 done
Classifying 25 Flowers images with NASNetMobile...
   → NASNetMobile done
Classifying 25 Flowers images with EfficientNetB0...
   → EfficientNetB0 done
Flowers classification completed and saved to:
   ../results/classifications/flowers_classifications.csv


Processing VGG16...
3/3 ━━━━━━━━━━━━━━━━━━━━ 8s 2s/step
   → Features extracted: (35, 512)
```



→ Plot saved: ../results/plots/VGG16_PCA_faces.png



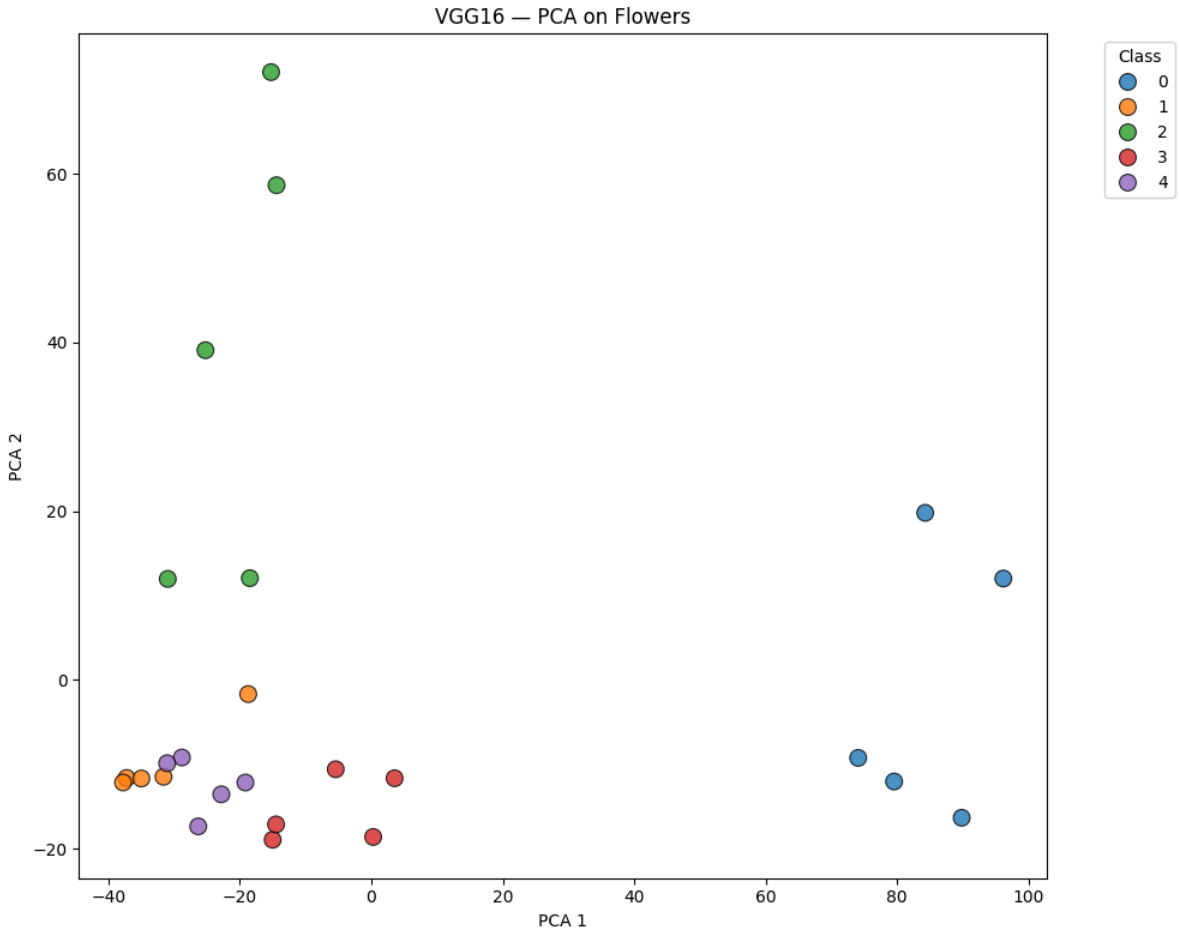
→ Plot saved: ../results/plots/VGG16_t-SNE_faces.png



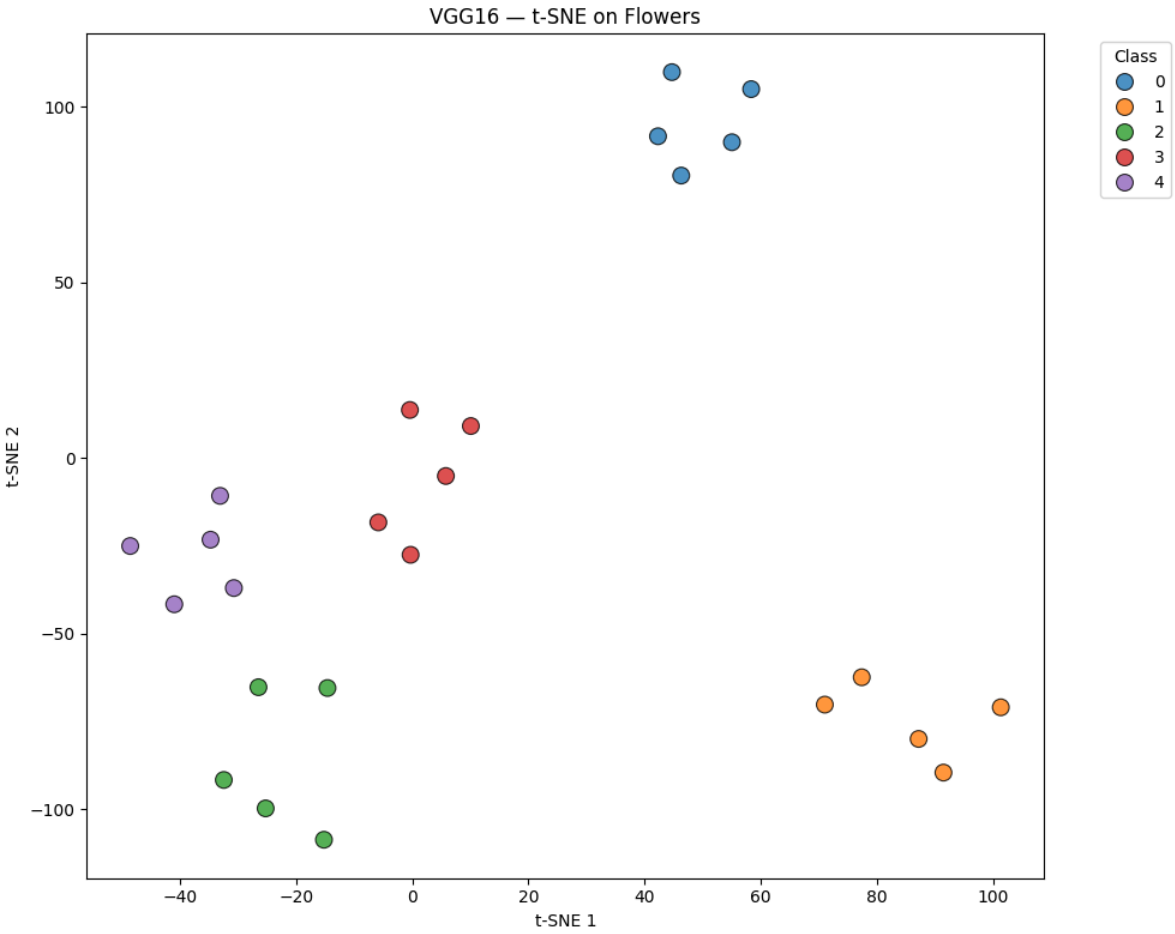
→ Plot saved: ../results/plots/VGG16_UMAP_faces.png

2/2 ━━━━━━━━━━━━━━━━━━━━ **6s** 2s/step

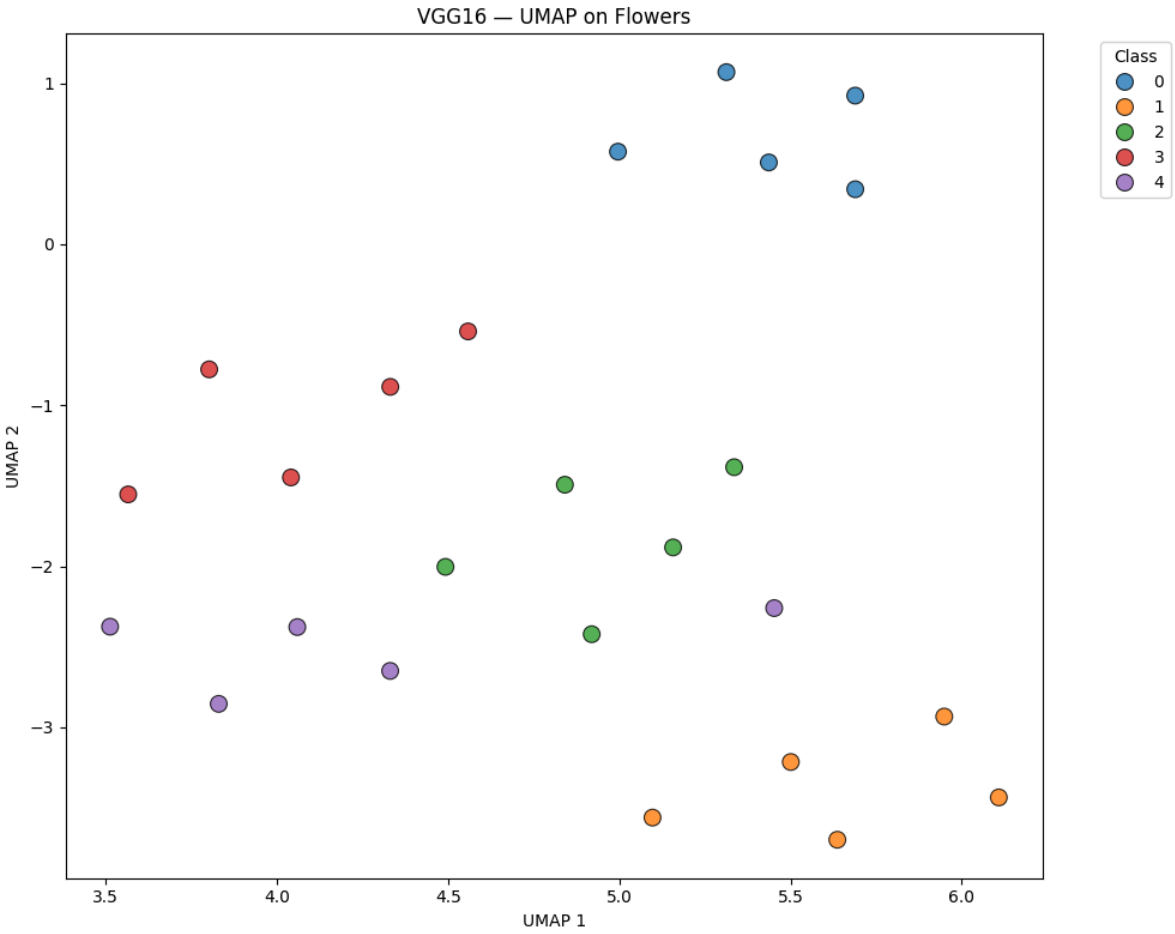
→ Features extracted: (25, 512)



→ Plot saved: ../results/plots/VGG16_PCA_flowers.png



→ Plot saved: ../results/plots/VGG16_t-SNE_flowers.png

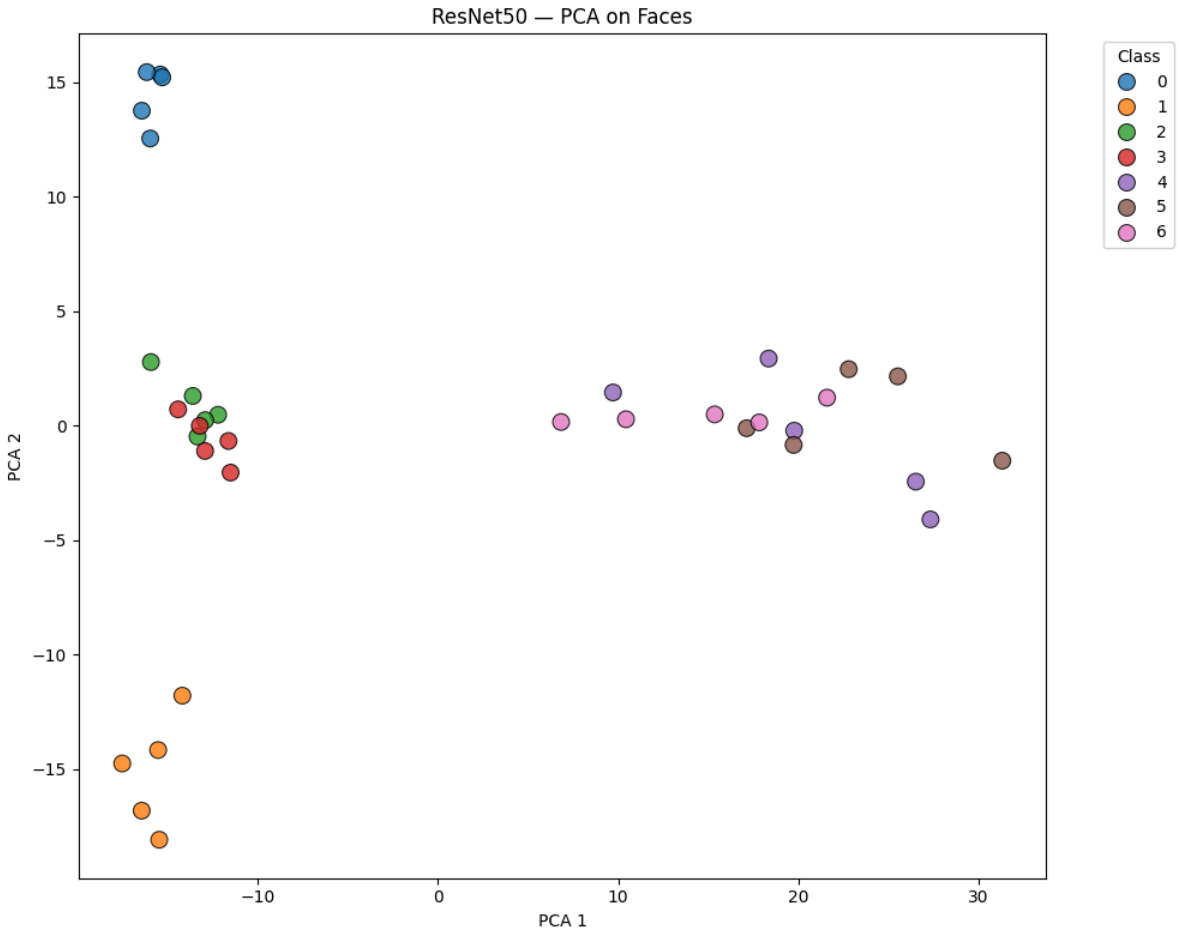


→ Plot saved: ../results/plots/VGG16_UMAP_flowers.png

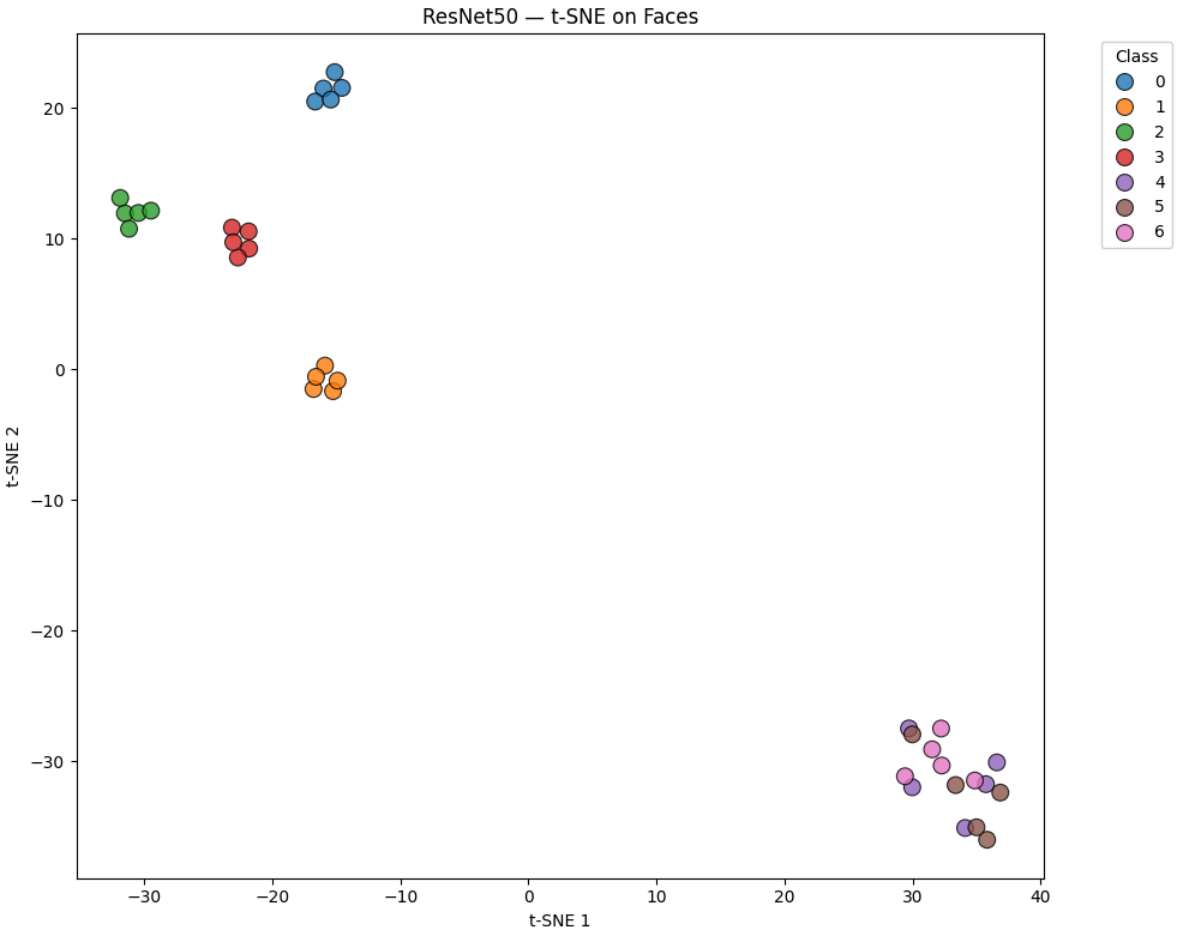
Processing ResNet50...

3/3 ━━━━━━━━━━━━━━━━━━━━ **5s** 2s/step

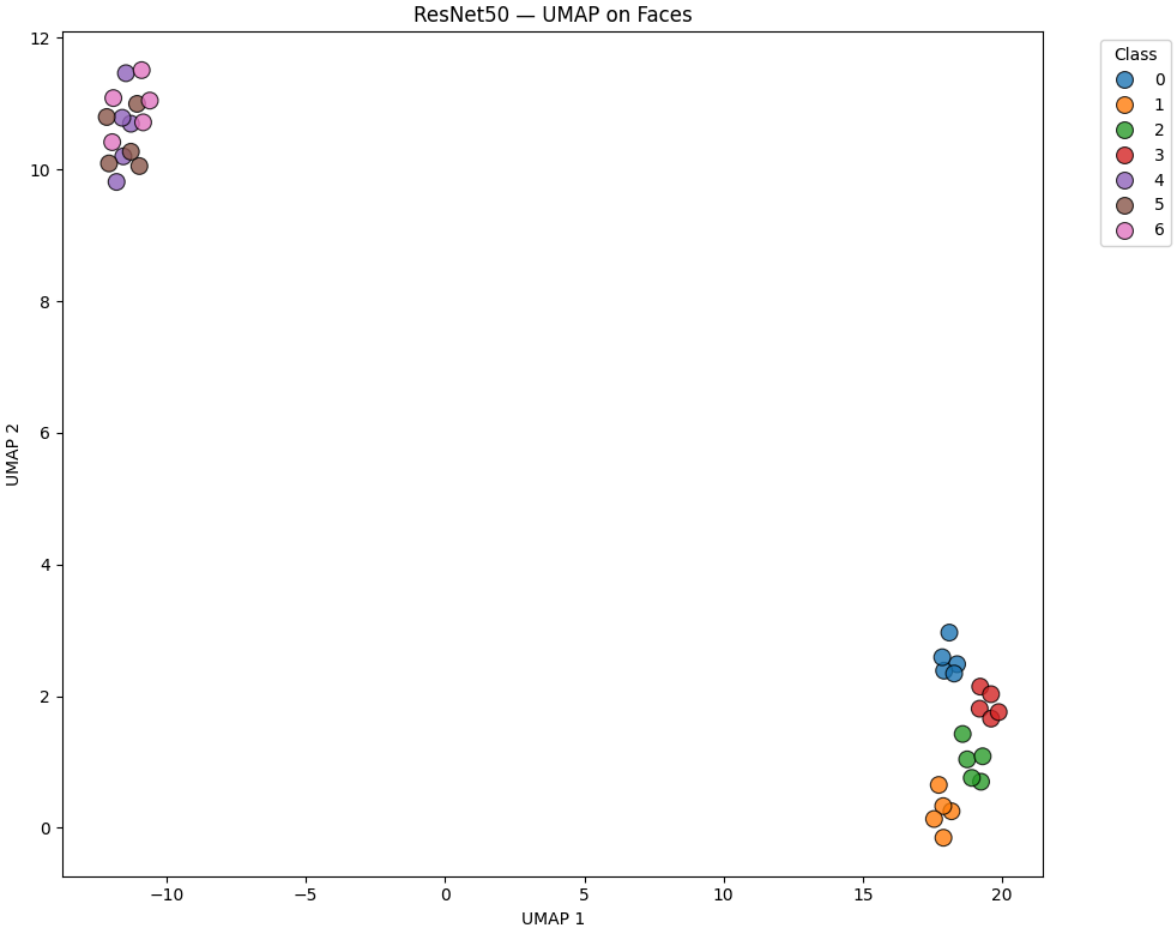
→ Features extracted: (35, 2048)



→ Plot saved: ../results/plots/ResNet50_PCA_faces.png



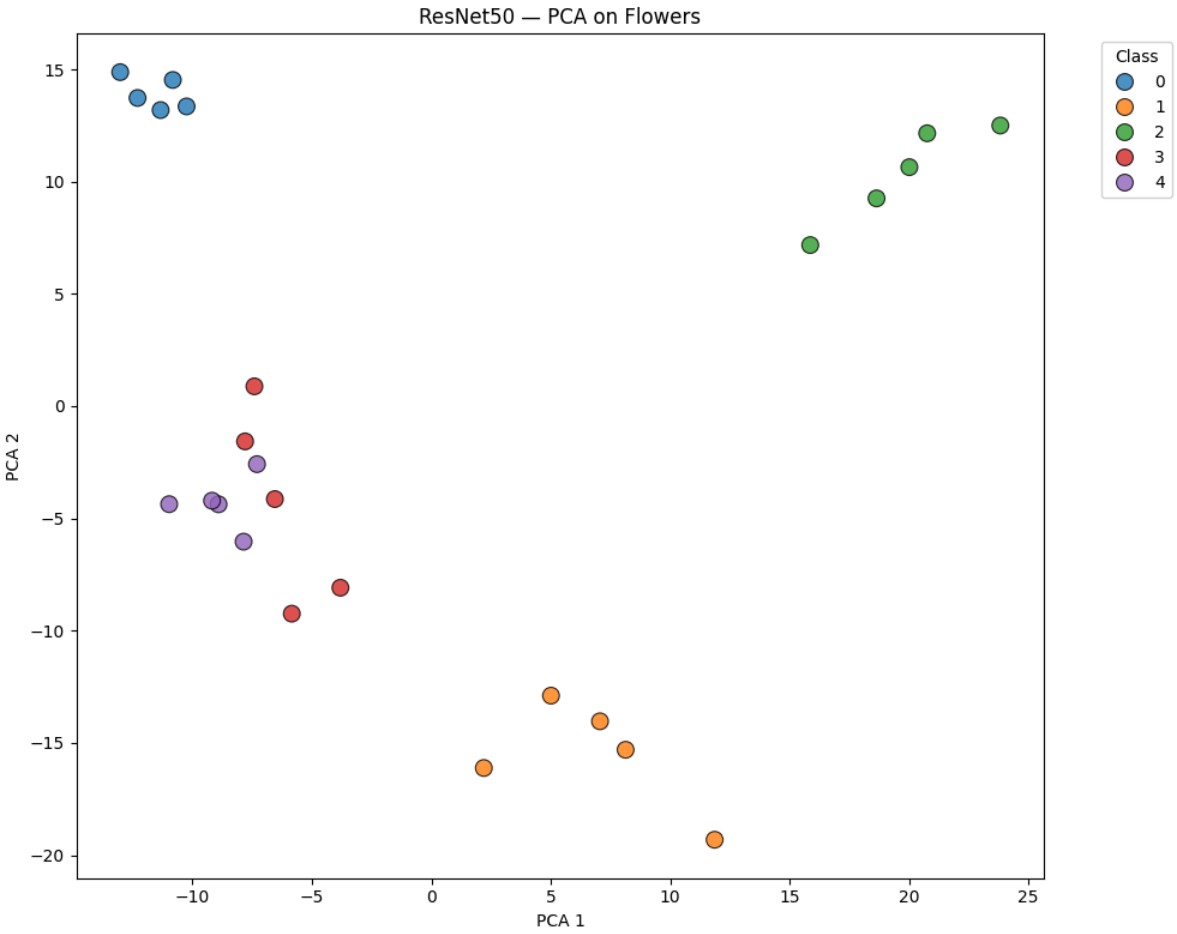
→ Plot saved: ../results/plots/ResNet50_t-SNE_faces.png



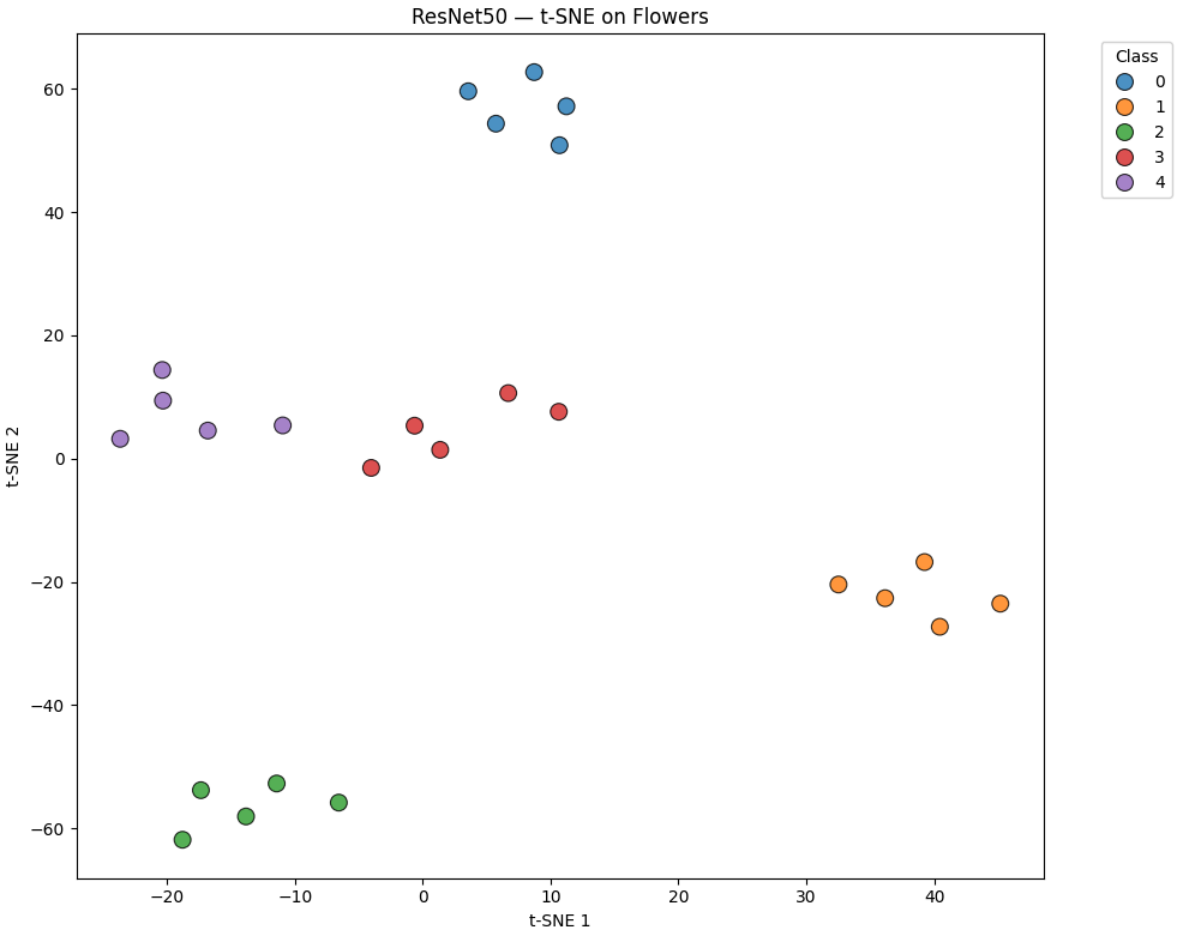
→ Plot saved: ../results/plots/ResNet50_UMAP_faces.png

2/2 ━━━━━━━━━━━━━━━━━━━━ **4s** 2s/step

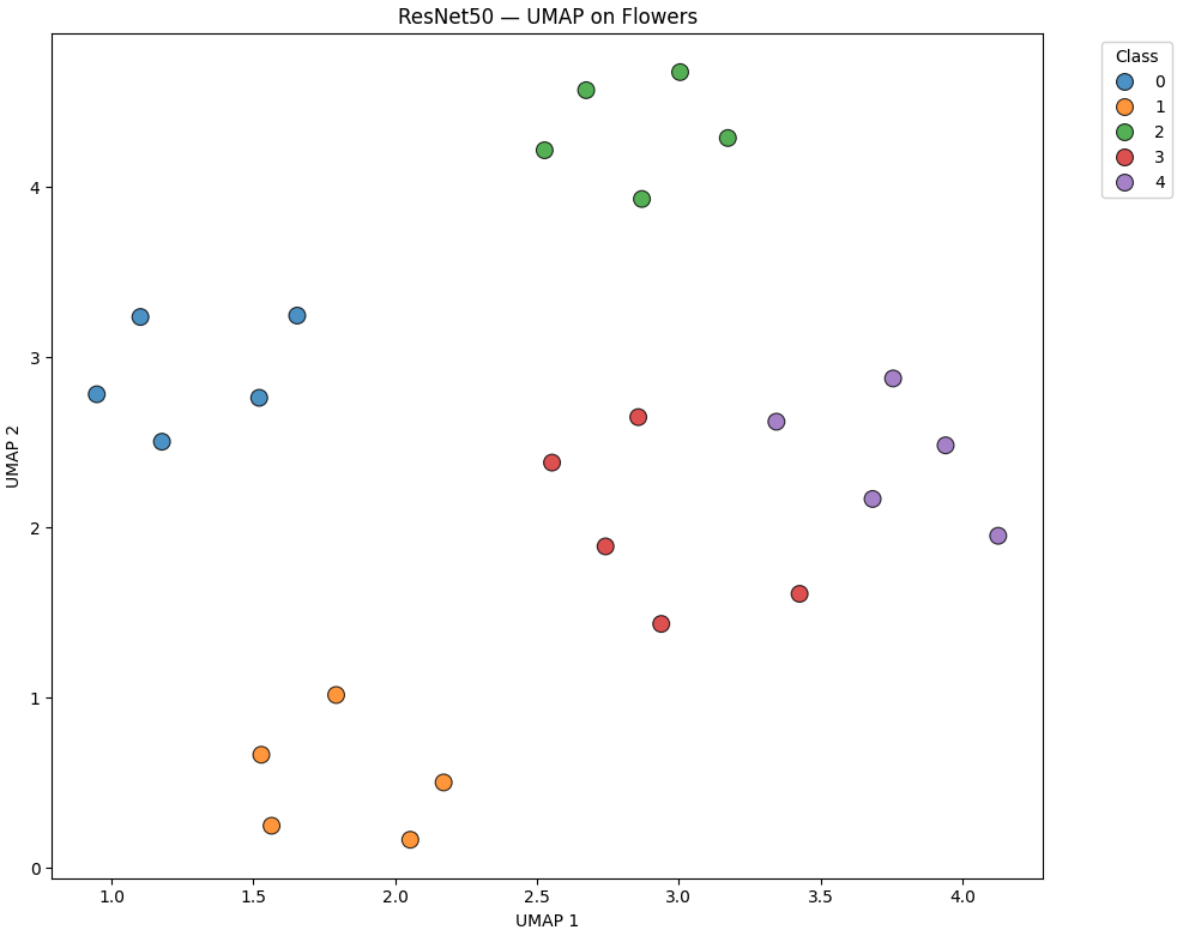
→ Features extracted: (25, 2048)



→ Plot saved: ../results/plots/ResNet50_PCA_flowers.png



→ Plot saved: ../results/plots/ResNet50_t-SNE_flowers.png

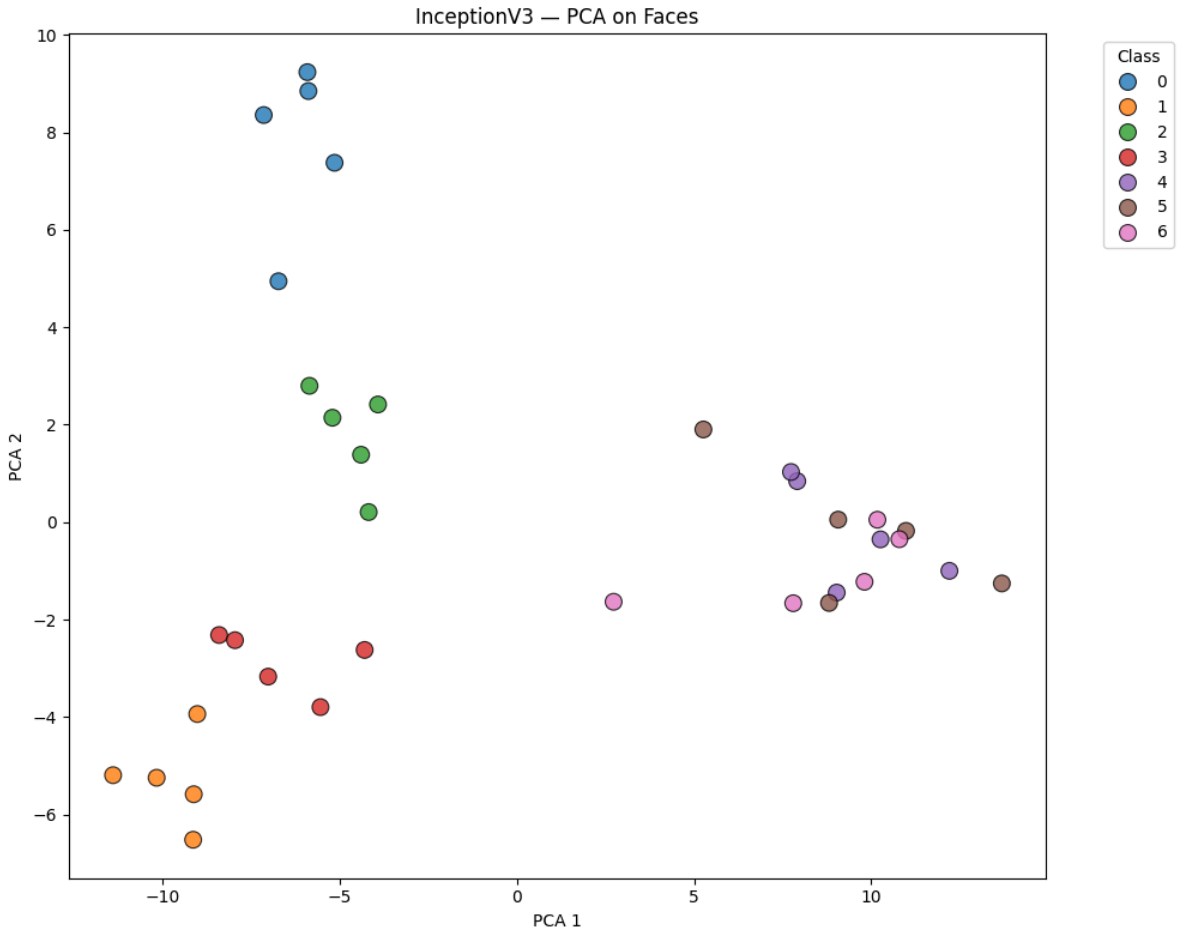


→ Plot saved: ../results/plots/ResNet50_UMAP_flowers.png

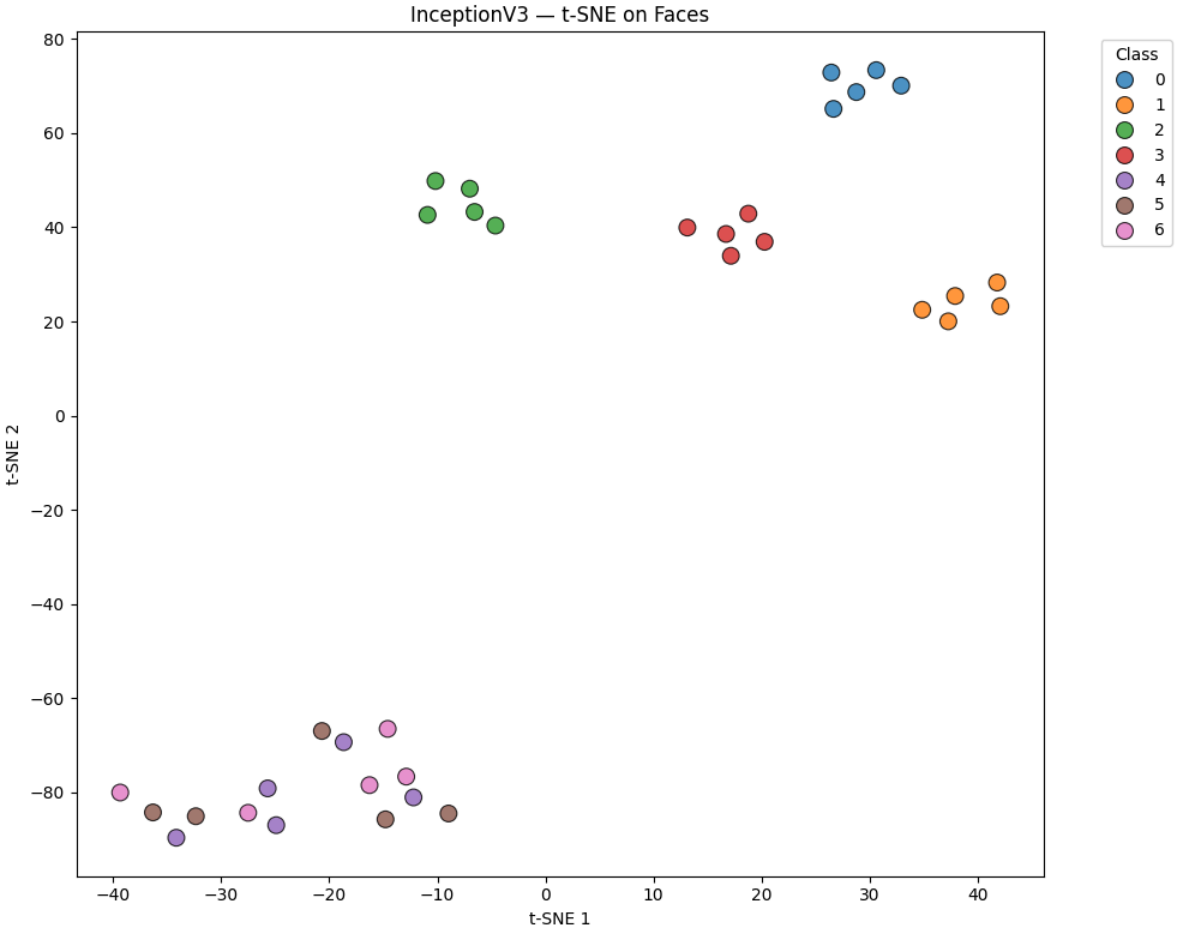
Processing InceptionV3...

3/3 ━━━━━━━━━━━━━━━━━━━━ **9s** 3s/step

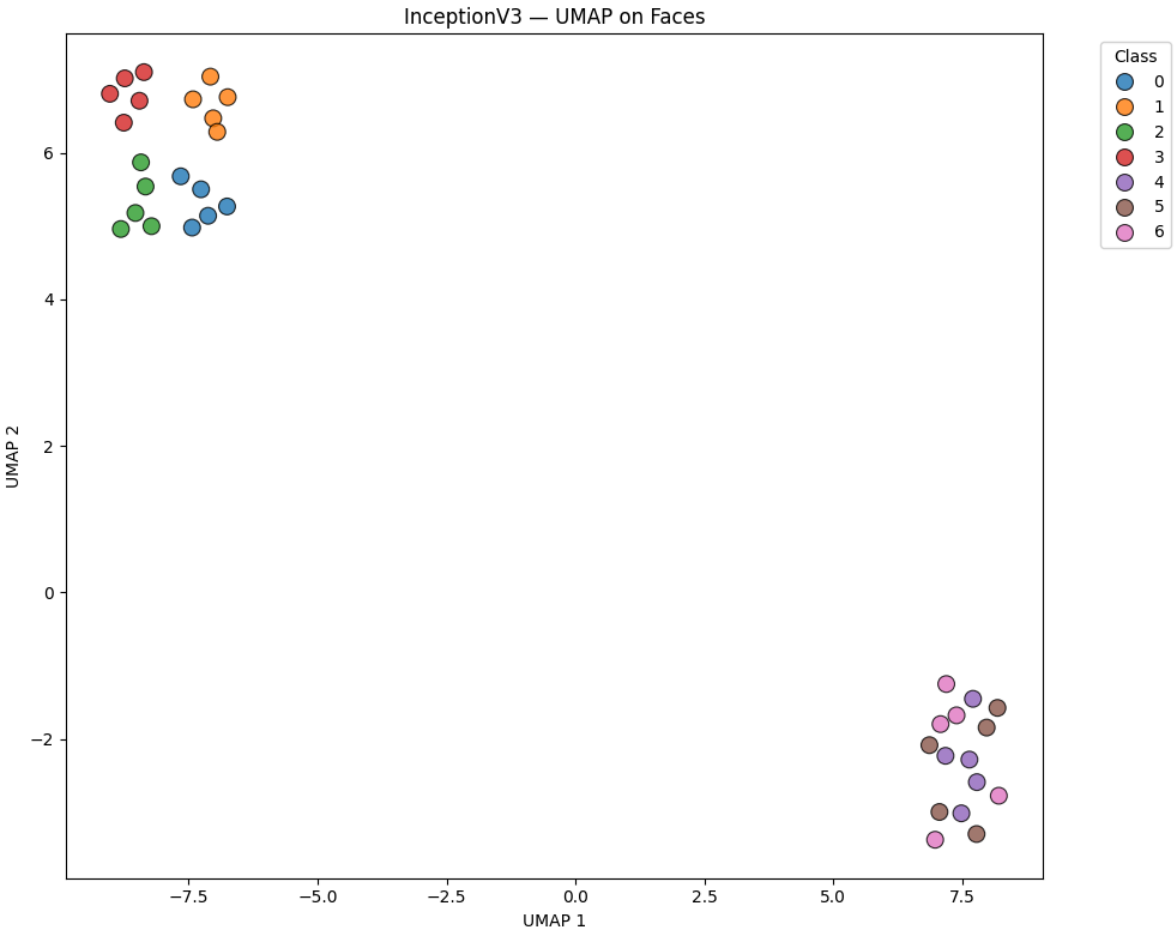
→ Features extracted: (35, 2048)



→ Plot saved: ../results/plots/InceptionV3_PCA_faces.png



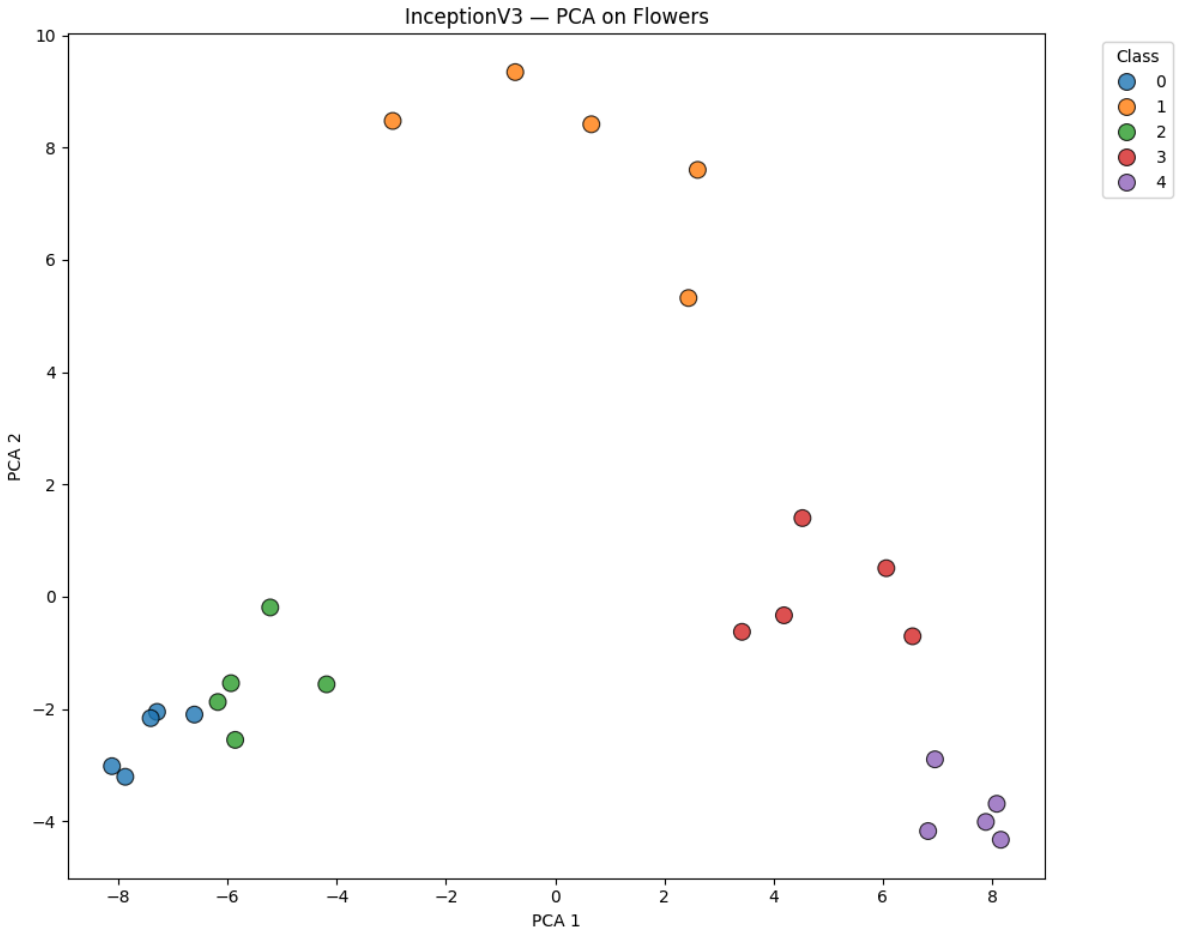
→ Plot saved: ../results/plots/InceptionV3_t-SNE_faces.png



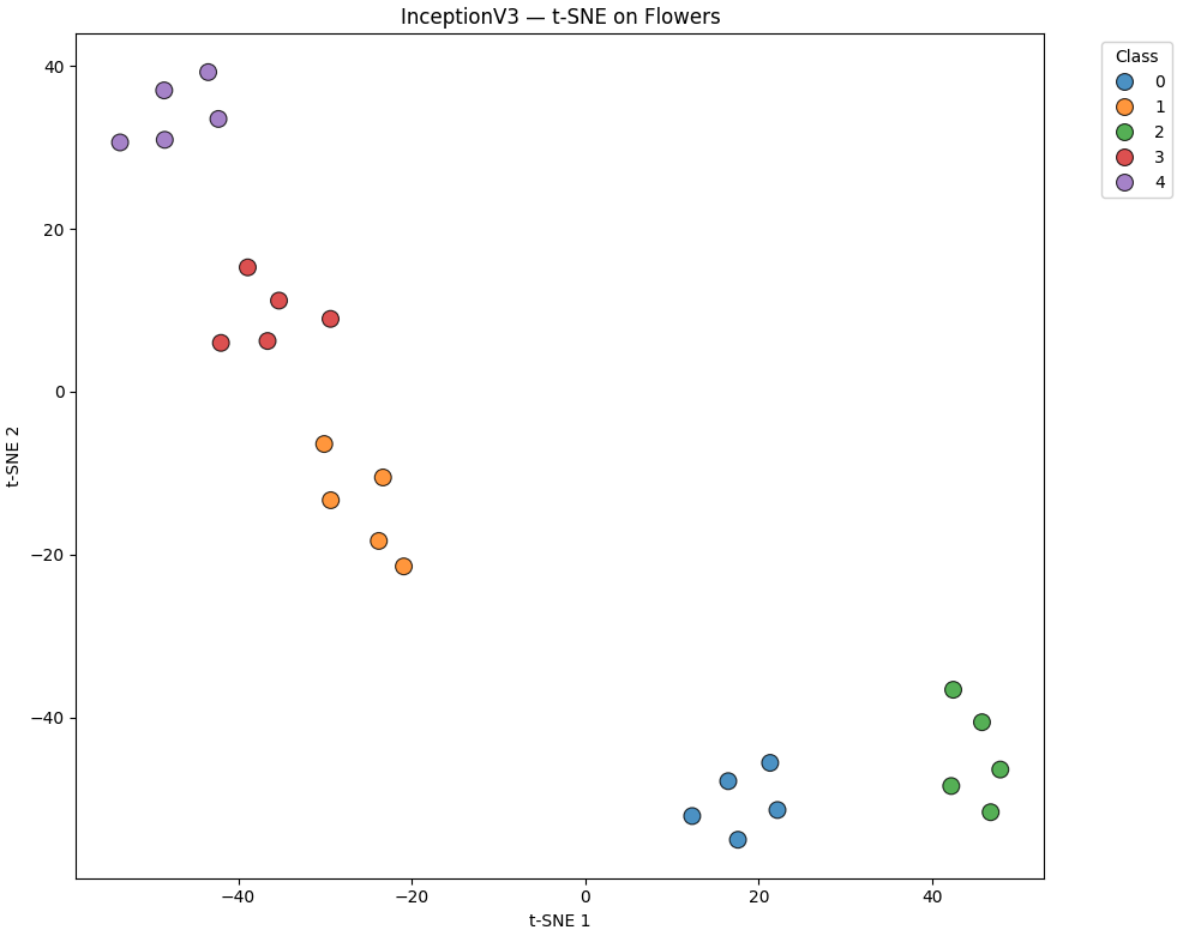
→ Plot saved: ../results/plots/InceptionV3_UMAP_faces.png

2/2 ━━━━━━━━━━━━━━━━━━━━ **6s** 3s/step

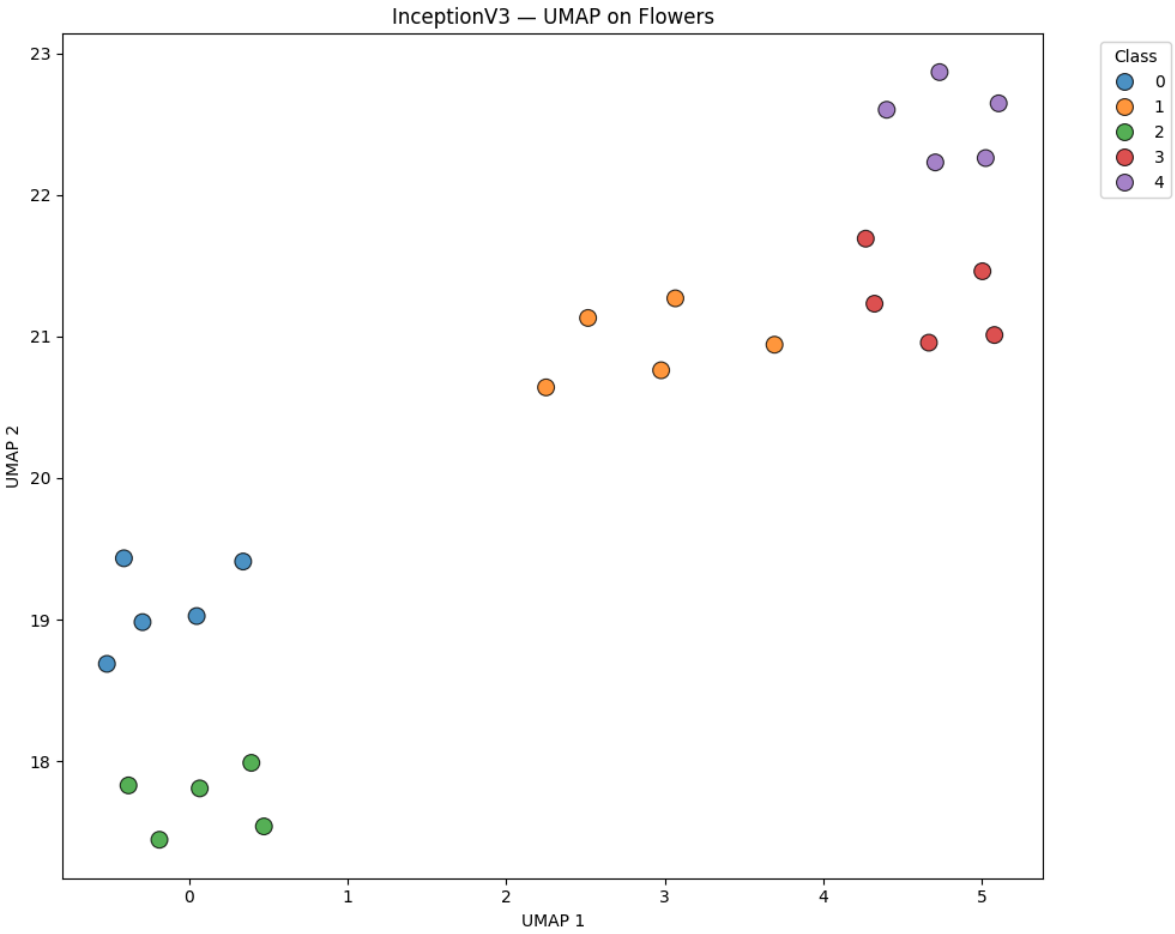
→ Features extracted: (25, 2048)



→ Plot saved: ../results/plots/InceptionV3_PCA_flowers.png



→ Plot saved: ../results/plots/InceptionV3_t-SNE_flowers.png

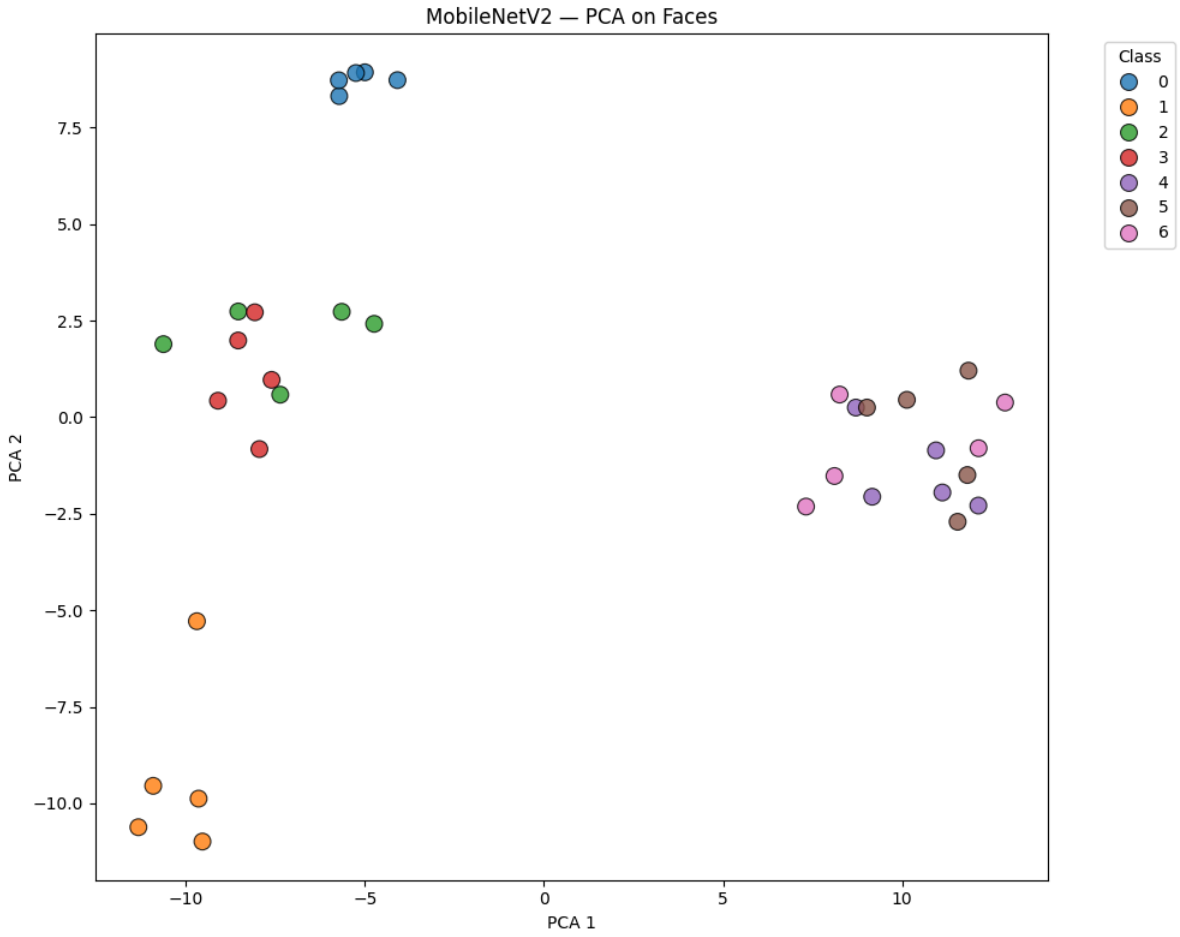


→ Plot saved: ../results/plots/InceptionV3_UMAP_flowers.png

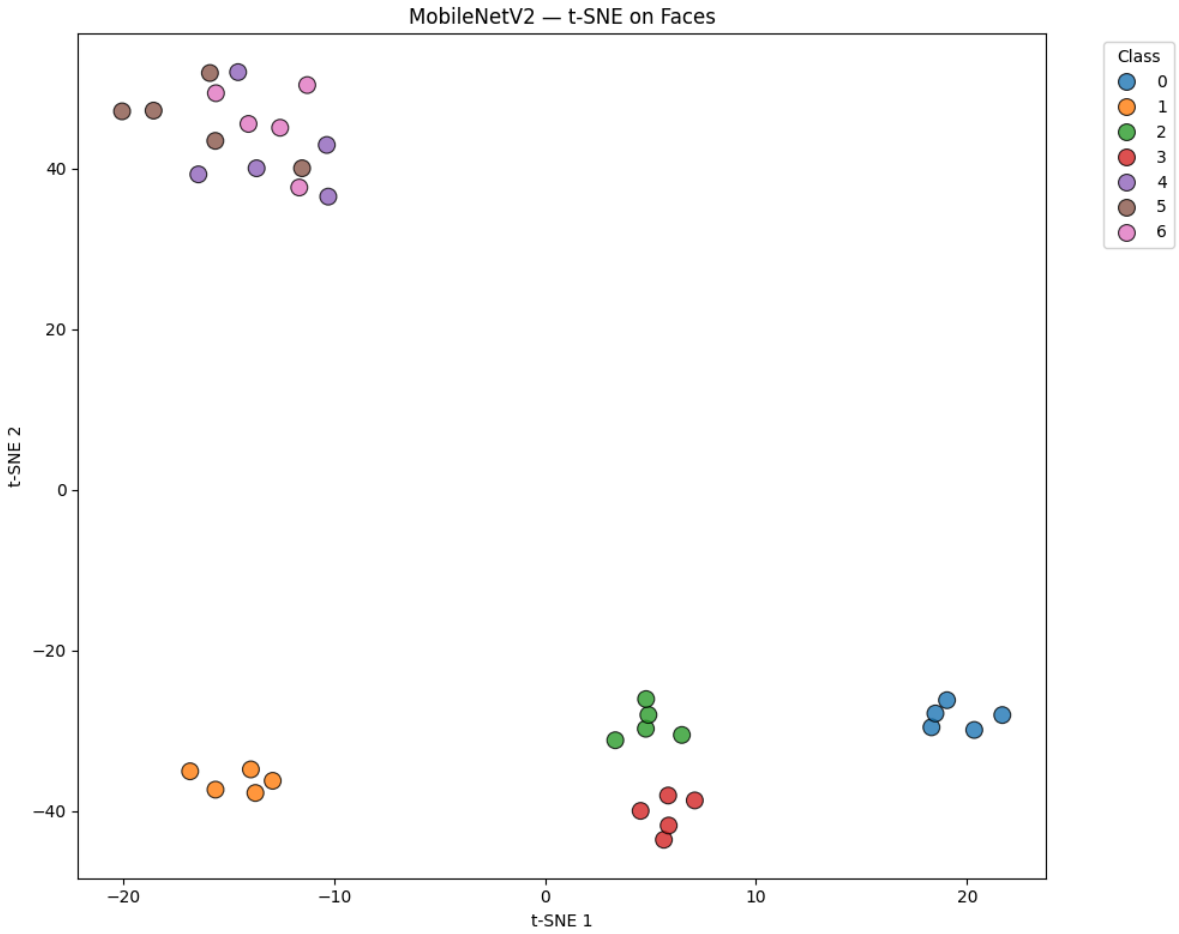
Processing MobileNetV2...

3/3 ━━━━━━━━━━━━━━━━━━━━ **2s** 637ms/step

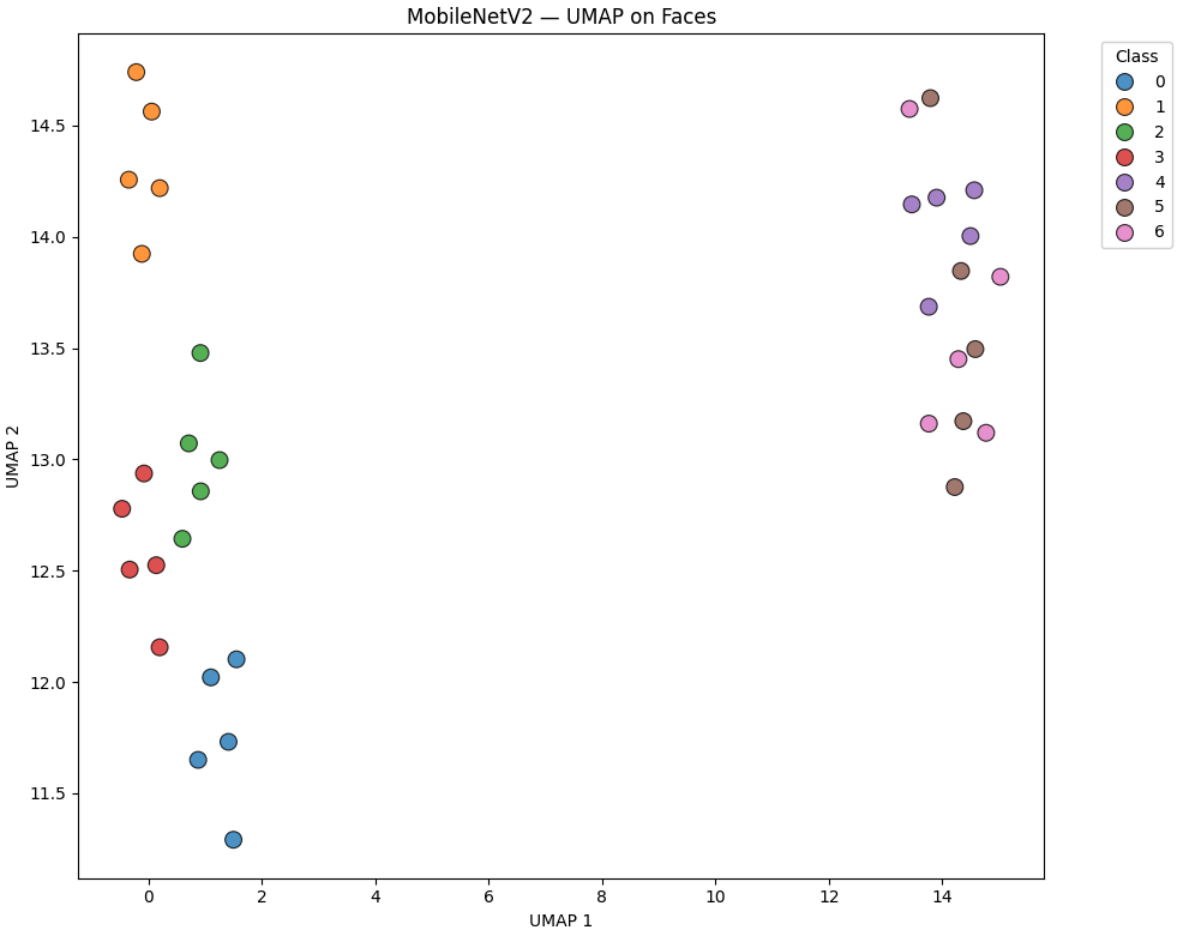
→ Features extracted: (35, 1280)



→ Plot saved: ../results/plots/MobileNetV2_PCA_faces.png



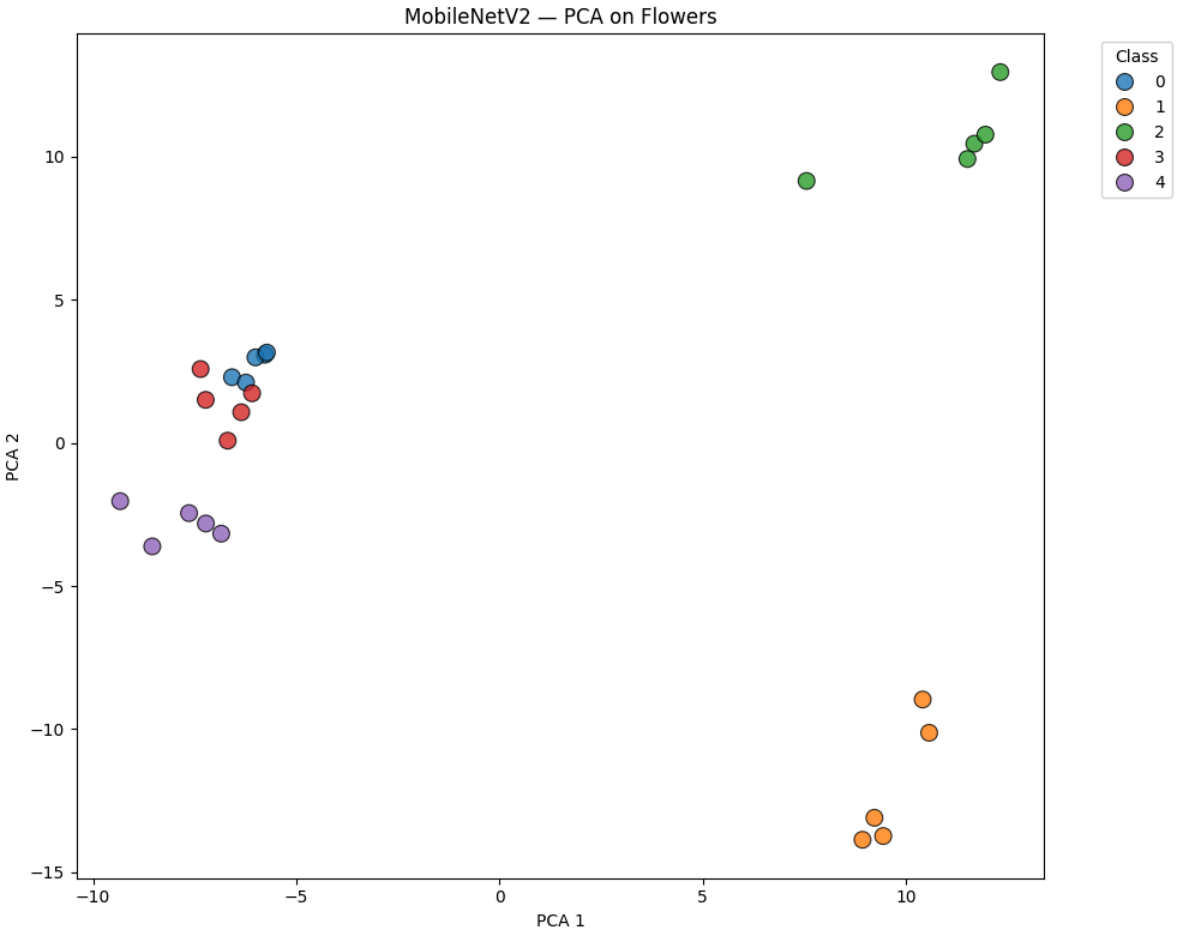
→ Plot saved: ../results/plots/MobileNetV2_t-SNE_faces.png



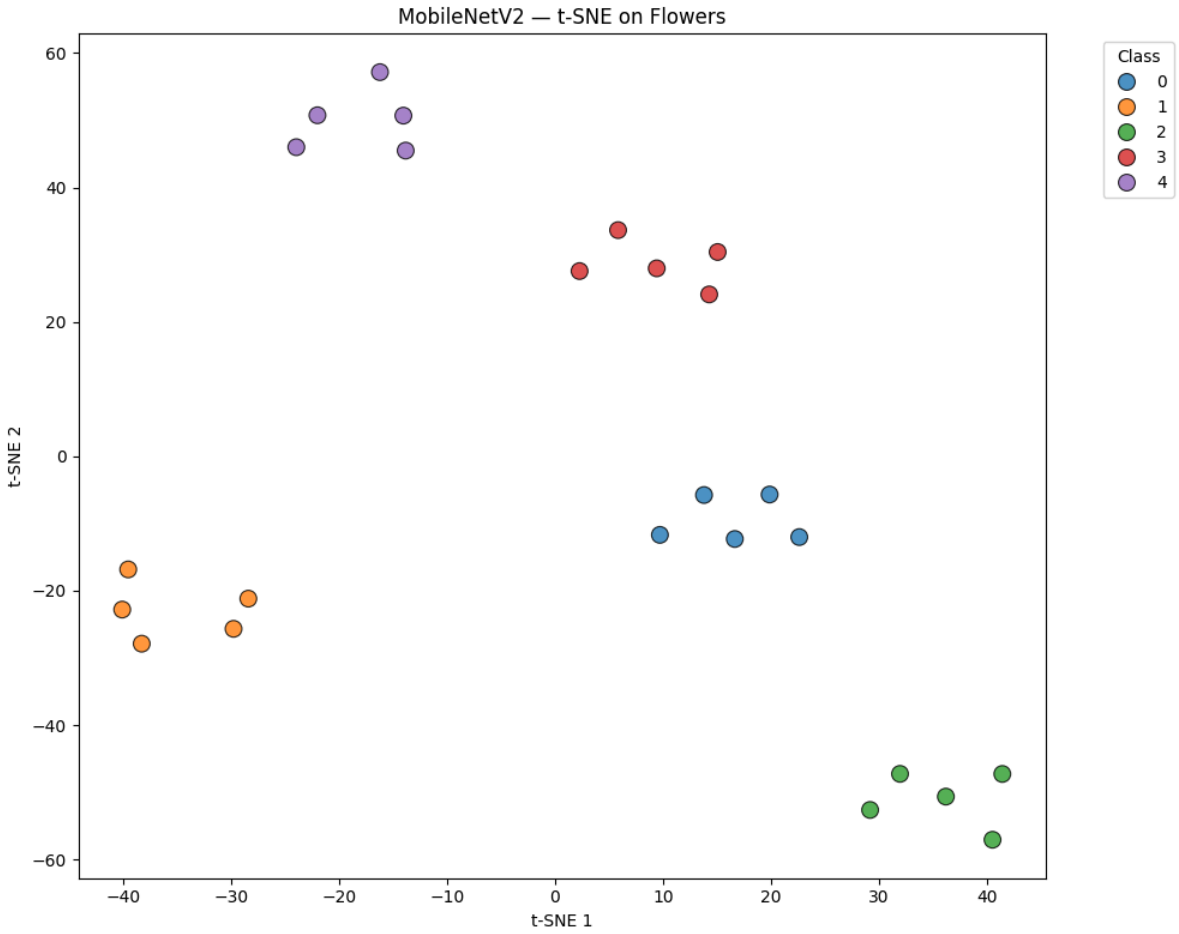
→ Plot saved: ../results/plots/MobileNetV2_UMAP_faces.png

2/2 ━━━━━━━━━━━━━━━━━━━━ **3s** 1s/step

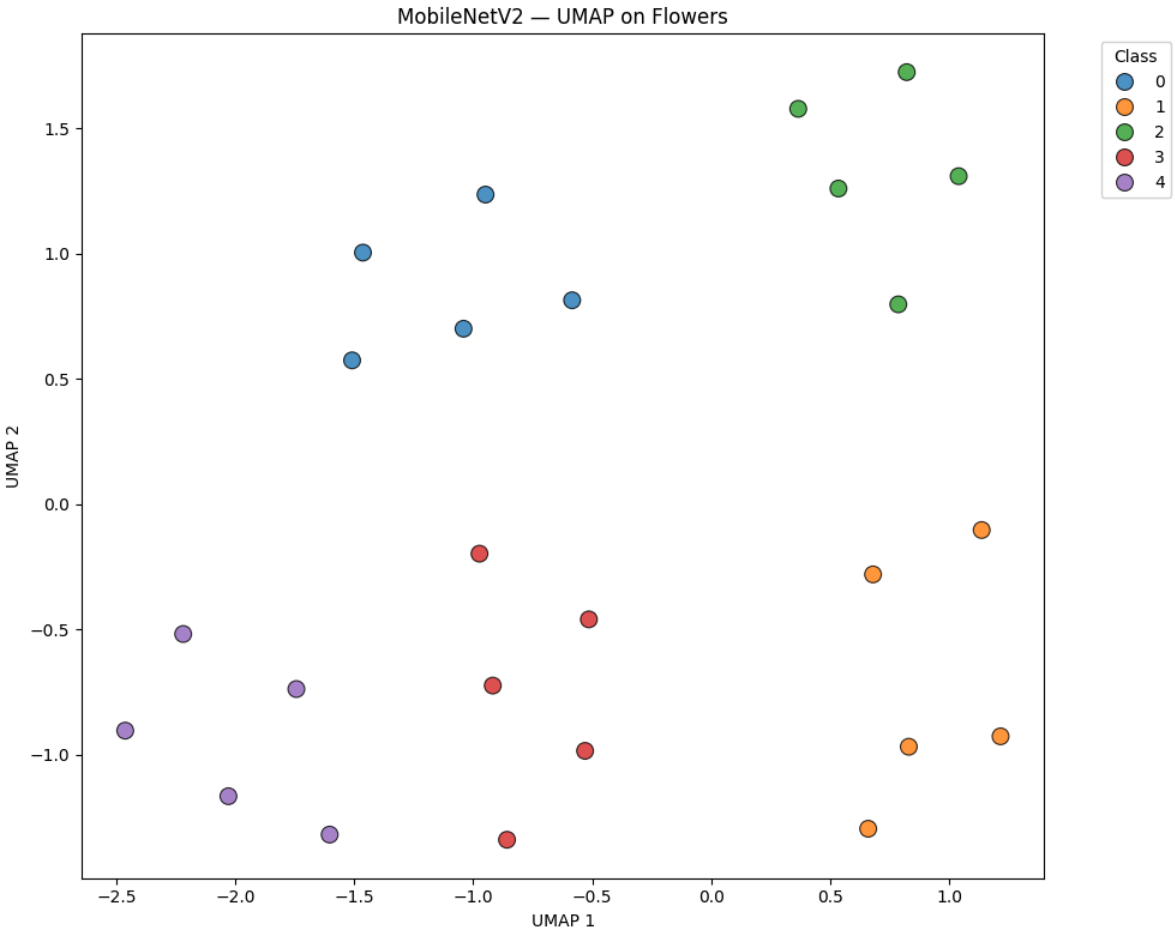
→ Features extracted: (25, 1280)



→ Plot saved: ../results/plots/MobileNetV2_PCA_flowers.png



→ Plot saved: ../results/plots/MobileNetV2_t-SNE_flowers.png

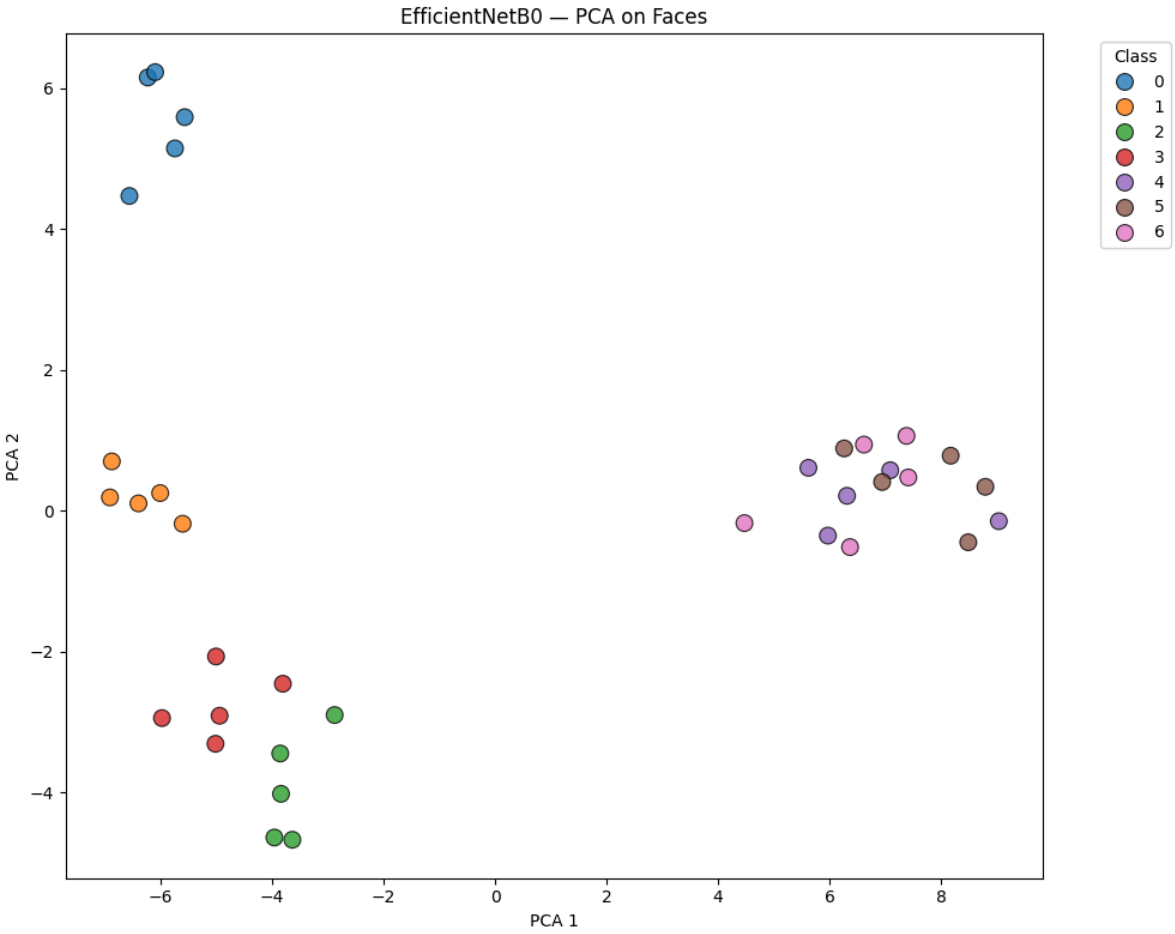


→ Plot saved: ../results/plots/MobileNetV2_UMAP_flowers.png

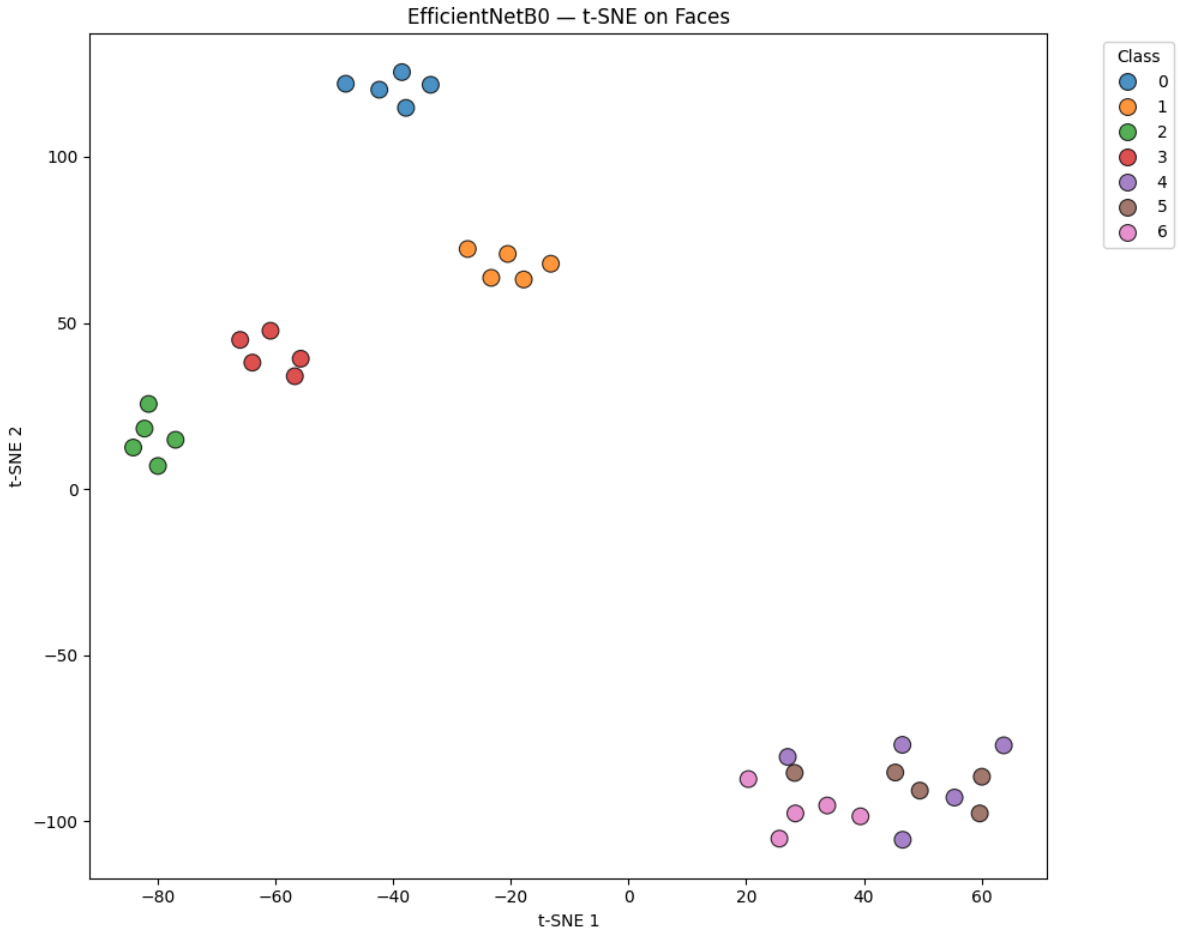
Processing EfficientNetB0...

3/3 ━━━━━━━━━━━━━━━━━━━━ **5s** 1s/step

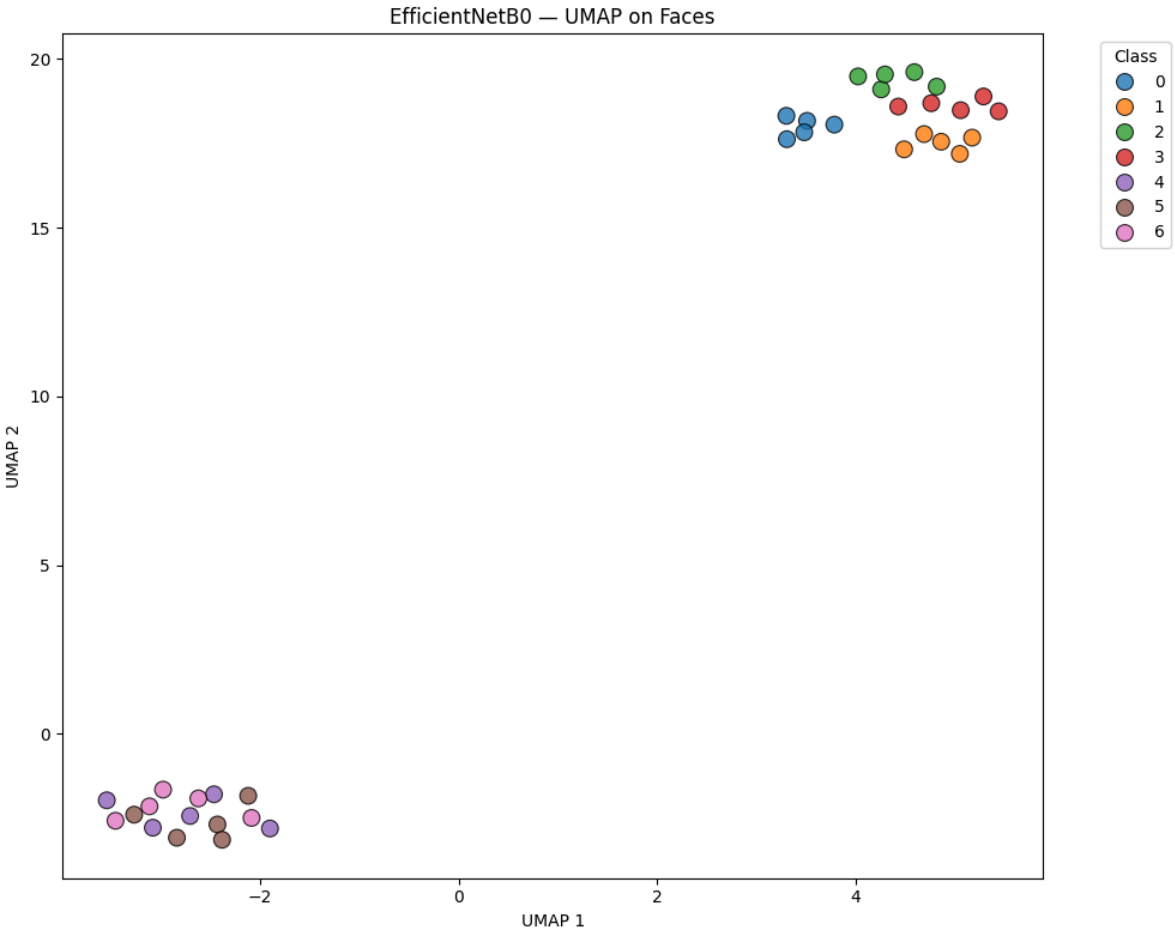
→ Features extracted: (35, 1280)



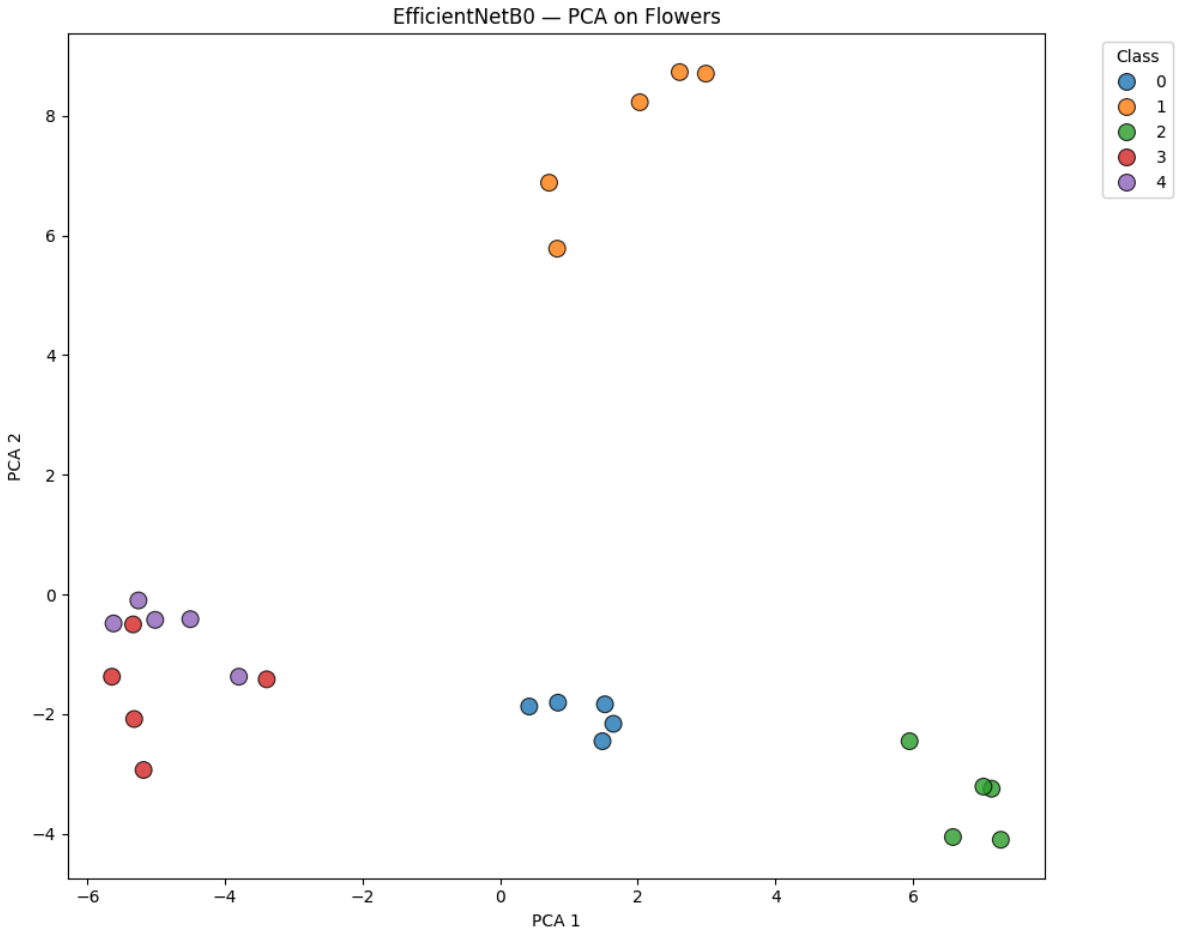
→ Plot saved: ../results/plots/EfficientNetB0_PCA_faces.png



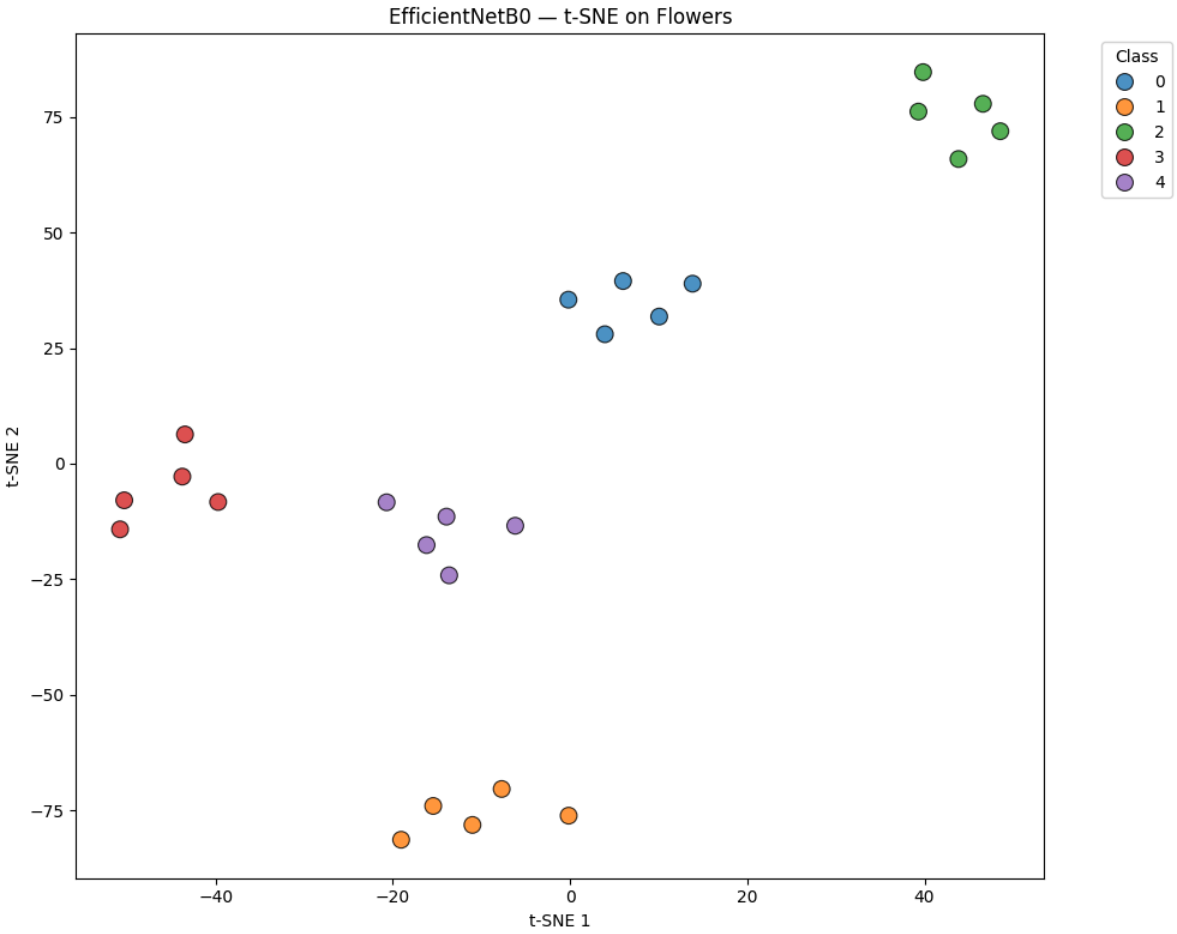
→ Plot saved: ../results/plots/EfficientNetB0_t-SNE_faces.png



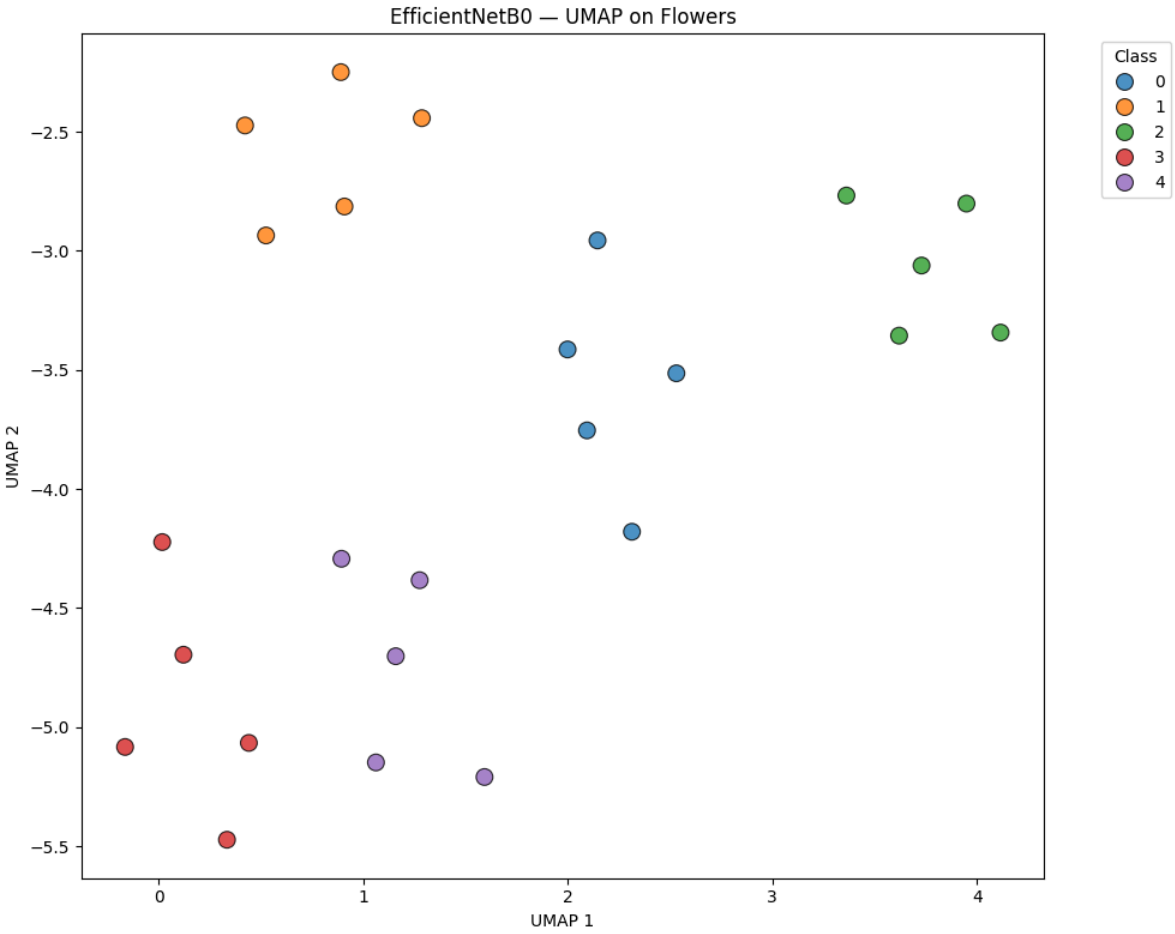

```
   → Plot saved: ../results/plots/EfficientNetB0_UMAP_faces.png
2/2 ━━━━━━━━━━━━━━━━━━━━ 6s 3s/step
   → Features extracted: (25, 1280)
```



→ Plot saved: ../results/plots/EfficientNetB0_PCA_flowers.png



→ Plot saved: ../results/plots/EfficientNetB0_t-SNE_flowers.png



→ Plot saved: ../results/plots/EfficientNetB0_UMAP_flowers.png

Pipeline completed! All plots saved in results/plots/